Stacklyst

Casos de uso e modelos do sistema

Laboratório de Engenharia de Software
Revisão de 3 de novembro de 2026
Base de código 6a2bfce e documentação local revisada

Pedro Antonio Silvestre Tescaro
Guilherme Alves de Campos
Paulo Henrique Leal dos Santos

Esta edição atualiza os modelos e a especificação do sistema. Os diagramas de atividades do negócio aprovados permanecem no documento 02, preservado integralmente. Os modelos revisados são editáveis no Astah UML. Cadastro e acesso usam o ator Usuário e declaram as pré-condições de sessão em cada caso.

As afirmações descrevem a fonte de implementação. Credenciais, infraestrutura externa, dispositivos e migrations dependem das verificações de ambiente indicadas. Esta revisão documental não representa execução de todos os fluxos em produção.

Roteiro de leitura

Roteiro de leitura 2
Referência de UML e critérios de modelagem 5
Casos de uso e sua especificação 5
Herança dos atores 5
Inclusão, extensão e pontos de extensão 5
Classes e persistência 6
Sequências e ligação com o código 6
Uso no seminário 6
Catálogo de casos de uso 7
UC001 Cadastrar Conta 8
UC002 Fazer Login 9
UC003 Gerenciar Perfil 10
UC004 Consultar Trilhas de Estudo 11
UC005 Realizar Lição 12
UC006 Resolver exercício 13
UC007 Emitir Classificação e Desempenho 14
UC008 Realizar Duelo 15
UC010 Enviar Solução de Duelo 16
UC012 Consultar Ajuda de Exercício 17
UC013 Gerenciar Publicações 18
UC014 Registrar Interação 19
UC015 Gerenciar Eventos 20
UC016 Avaliar Duelo 21
UC017 Gerenciar Empresas 22
UC018 Gerenciar Vagas 23
UC019 Gerenciar Usuários 24
UC021 Notificar Usuário 25
UC022 Registrar Candidatura 26
UC023 Criar evento 27

UC024 Apresentar Pedido de Credenciamento 28
UC025 Gerenciar Candidaturas 29
UC026 Avaliar Resposta 30
UC027 Executar Código e Testes 31
UC028 Gerenciar Denúncias 32
UC029 Fazer Denúncia 33
Caso de Uso Geral 34
Classes principais do Stacklyst 35
Visão relacional geral reduzida 36
Classes de aprendizagem 37
Dados de aprendizagem 38
Classes de comunidade 39
Dados de comunidade 40
Classes de duelos 41
Dados de duelos 42
Classes de recrutamento 43
Dados de recrutamento 44
Classes de eventos 45
Dados de eventos 46
Classes de identidade 47
Dados de identidade 48
Sequência completa 1 49
Sequência completa 2 50
Sequência completa 3 51
Sequência completa 4 52
Sequência completa 5 53
Sequência completa 9 54
Sequência completa 10 55
Sequência completa 11 56
Sequência completa 12 57
Sequência completa 13 58

Receber e avaliar resposta 59
Separar prévia de submissão 60
Creditar primeira conclusão 61
Autorizar publicação de vaga 62
Gravar vaga e etapas 63
Confirmar publicação 64
Registrar candidatura 65
Conferir gestão da candidatura 66
Atualizar e notificar 67
Correspondência dos modelos com a implementação 68
Tabelas relacionais 69
Uso da documentação no seminário 69
O que foi aproveitado do material fornecido 69
Ordem lógica sugerida 70
Roteiro de cada funcionalidade 70
Diagramas gerais e específicos 71
Recortes de sequência e código 71
Dados preparados 71
Tempo e divisão 72
Checklist de entrega 72
Leitura do diagrama de classes geral 73
Interfaces atuais dos fluxos apresentados 74
Realizar Lição 74
Resolver exercício 74
Vagas e recrutamento 75
Detalhes da vaga e acompanhamento da candidatura 75
Eventos e hackathons 76
Avaliação de código e pedido de credenciamento 76

O sumário é atualizado na exportação pelo Word. As fontes editáveis e a matriz de rastreabilidade acompanham o pacote.

Revisão de 3 de novembro de 2026 4

Referência de UML e critérios de modelagem

Referência principal: GUEDES, Gilleanes T. A. UML 2: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018. ISBN 978-85-7522-644-5. Consultado no PDF fornecido pelo grupo. Os números de página abaixo são a posição no PDF de 657 páginas; podem diferir da paginação impressa.

O livro orienta a notação e a organização da documentação. As regras da Stacklyst são verificadas no código do projeto; os exemplos bancários e demais sistemas do livro não são requisitos desta plataforma. Os textos e diagramas desta revisão são próprios do projeto.

Casos de uso e sua especificação

Conforme a seção 3.4 (PDF, pp. 65-67), um caso de uso deve explicar o objetivo do ator e a resposta do sistema em linguagem compreensível. O catálogo identifica nome e ID, ator principal, pré-condições, fluxo principal, resultado esperado e alternativas. Referências de código e regras de negócio complementam a especificação e permitem verificar a implementação. A UML admite formatos diferentes para essa documentação; a estrutura adotada é uma adaptação ao projeto.

Cada passo do catálogo deve ser lido como interação entre ator e sistema. No diagrama, as elipses representam objetivos ou comportamentos reutilizados. Tabelas, componentes React, rotas e consultas são detalhados nos modelos estruturais e nas sequências, sem transformá-los em atores humanos.

Herança dos atores

A generalização/especialização segue a seção 3.7 (PDF, pp. 70-73). Avaliador, Recrutador e Administrador especializam Usuário. A linha contínua termina em triângulo vazio voltado para Usuário, o ator geral. Um papel especializado participa das interações comuns de Usuário e acrescenta as interações autorizadas ao seu papel.

Essa generalização representa papéis de interação. Não afirma que existam subclasses TypeScript desses nomes nem concede permissões em execução: o servidor continua validando sessão, papel, propriedade e escopo. Uma mesma pessoa pode exercer mais de um papel. Cadastro, recuperação e verificação pública são interações de Usuário cujas pré-condições permitem ausência de sessão.

Inclusão, extensão e pontos de extensão

Na inclusão, o comportamento chamado é obrigatório no contexto do caso que o inclui (seção 3.8; PDF, pp. 74-75). A dependência tracejada com «include» aponta do caso base para o incluído. Na visão geral, Resolver exercícios inclui Avaliar resposta; Submeter solução de duelo inclui Executar código. A validação das entradas e a autorização antecedem a execução efetiva, conforme os fluxos alternativos documentados.

Na extensão, o comportamento adicional ocorre sob uma condição, em um ponto identificável do caso base (seções 3.9 a 3.11; PDF, pp. 76-80). A dependência tracejada com «extend» aponta da extensão para o caso base. Executar código estende Avaliar resposta quando a atividade é de código, no ponto avaliar código. Consultar ajuda estende Resolver exercícios se o usuário solicitar e o modo permitir, no ponto consultar ajuda. Atividades objetivas podem ser avaliadas sem execução de código; a ajuda não é uma etapa obrigatória de todos os exercícios.

Relações de inclusão e extensão descrevem dependência comportamental. Não indicam ordem cronológica de navegação. Autenticação é pré-condição dos fluxos protegidos; não se repete um «include Fazer login» para toda operação.

Classes e persistência

Os capítulos 2 e 4 fundamentam classes, atributos, operações e multiplicidades. A seção 4.15 (PDF, pp. 213-218) distingue classes de tabelas e discute o mapeamento para persistência. As vistas CL desta revisão são conceituais, com nomes em português e responsabilidades do domínio. A visão geral tem oito classes; as vistas específicas têm quatro a seis. Uma classe conceitual não afirma que exista uma declaração TypeScript class com o mesmo nome.

As vistas DER são recortes relacionais em notação de classes UML: dez tabelas na visão geral e quatro ou cinco nas vistas específicas. CP significa chave primária e CE, chave estrangeira. Os nomes visíveis são traduções; a tabela de correspondência identifica as tabelas e campos reais. Campos e relações auxiliares permanecem no dicionário completo do banco. O Astah UML instalado não oferece o editor ER da edição Professional; a notação utilizada está explícita. A redução é de apresentação, sem alteração do schema.

Sequências e ligação com o código

O capítulo 7 orienta participantes, linhas de vida, mensagens, retornos e condições. As seções 7.3.3 e 7.8 (PDF, pp. 272-273 e 286-293) apoiam a leitura de respostas e alternativas. A ordem vertical mostra a progressão da interação; a posição horizontal separa responsabilidades.

Nas vistas nativas desta revisão, chamadas são mensagens UML editáveis e condições são guardas nas mensagens e notas de resposta. As respostas podem ser expressas nos valores de retorno das chamadas e em notas de resultado, dispensando setas de retorno redundantes. As sequências completas devem ser lidas junto ao fluxo textual e suas alternativas. Os recortes SQ01-A a SQ01-C e SQ11-A a SQ11-F relacionam uma parte da interação ao arquivo e às linhas reais da implementação.

O cenário principal, os caminhos alternativos e as falhas previstas precisam concordar com a especificação. Um retorno de erro não comprova sucesso, uma prévia não concede recompensa de submissão e uma repetição não deve ser documentada como novo crédito de XP. Essas são regras da implementação conferidas separadamente da referência bibliográfica.

Uso no seminário

A sequência de exposição segue a orientação da professora: protótipo, RF/RNF/RN, modelos estruturais, sequência completa, recortes com código e demonstração prática. A referência de Guedes fundamenta os modelos; a ordem, o tempo e a divisão dos integrantes provêm da orientação do seminário. Os diagramas de atividades previamente aprovados permanecem preservados.

Catálogo de casos de uso

Esta especificação descreve os 26 casos representados na vista Caso de Uso Geral. Os identificadores e nomes reproduzem a vista fornecida pelo grupo. Cada caso contém objetivo, atores, pré-condições, fluxo principal, alternativas, pós-condições e referências à implementação.

A nota de autenticação do diagrama aplica-se às operações protegidas. Cadastrar Conta e Fazer Login são portas de entrada e não dependem de sessão anterior. Permissões administrativas, de avaliador e de recrutador são verificadas no servidor e por vínculo com o recurso.

ID	Nome do caso de uso
UC001	Cadastrar Conta
UC002	Fazer Login
UC003	Gerenciar Perfil
UC004	Consultar Trilhas de Estudo
UC005	Realizar Lição
UC006	Resolver exercício
UC007	Emitir Classificação e Desempenho
UC008	Realizar Duelo
UC010	Enviar Solução de Duelo
UC012	Consultar Ajuda de Exercício
UC013	Gerenciar Publicações
UC014	Registrar Interação
UC015	Gerenciar Eventos
UC016	Avaliar Duelo
UC017	Gerenciar Empresas
UC018	Gerenciar Vagas
UC019	Gerenciar Usuários
UC021	Notificar Usuário
UC022	Registrar Candidatura
UC023	Criar evento
UC024	Apresentar Pedido de Credenciamento
UC025	Gerenciar Candidaturas
UC026	Avaliar Resposta
UC027	Executar Código e Testes
UC028	Gerenciar Denúncias
UC029	Fazer Denúncia

UC001 Cadastrar Conta

Objetivo: Criar uma identidade própria para acessar a plataforma.

Atores: Usuário

Pré-condições: Não possuir conta com o mesmo e-mail ou nome de usuário; fornecer os dados obrigatórios.

Fluxo principal

1. O usuário abre o cadastro e informa nome de usuário, e-mail e senha.

2. O sistema valida os campos e verifica os identificadores disponíveis.

3. O provedor de autenticação cria a identidade e o sistema registra os dados do perfil.

4. O sistema apresenta a orientação de confirmação ou de acesso.

Fluxos alternativos e exceções

A1 — E-mail ou nome já utilizado: o sistema informa o conflito, mantém os campos permitidos e solicita outro identificador. O fluxo retoma a validação.

A2 — Dados inválidos: o sistema aponta o campo e impede a criação até a correção.

A3 — Confirmação de e-mail exigida pelo ambiente: a conta aguarda a confirmação antes do acesso autenticado.

Pós-condição de sucesso: Identidade e perfil registrados, com situação de confirmação apresentada.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/api/auth/register/route.ts.

Regras: R01, R02.

UC002 Fazer Login

Objetivo: Estabelecer uma sessão autenticada e recuperar o acesso quando necessário.

Atores: Usuário

Pré-condições: Possuir conta válida para autenticação; cadastro e recuperação são acessíveis sem sessão.

Fluxo principal

1. O usuário abre o acesso e informa suas credenciais.

2. O provedor de autenticação verifica a identidade.

3. O sistema resolve a conta e estabelece a sessão válida.

4. A interface encaminha o usuário ao conteúdo autorizado.

Fluxos alternativos e exceções

A1 — Credenciais recusadas ou conta indisponível: o sistema não cria sessão e apresenta a orientação de acesso.

A2 — Esquecimento da senha: o titular solicita recuperação e segue o link válido enviado pelo provedor de autenticação.

A3 — Sessão expirada durante uma operação protegida: o usuário autentica novamente e retorna ao recurso autorizado.

Pós-condição de sucesso: Sessão válida estabelecida ou recuperação solicitada pelo fluxo próprio.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/api/auth/reset-password/route.ts, src/lib/auth.ts.

Regras: R01, R02, R03.

UC003 Gerenciar Perfil

Objetivo: Consultar e atualizar seu perfil, aparência e preferências pessoais.

Atores: Usuário

Pré-condições: Sessão válida; alterações permitidas somente ao titular ou a papel autorizado pelo recurso.

Fluxo principal

1. O usuário abre seu perfil ou as preferências e escolhe os dados a alterar.

2. A interface apresenta os dados e as personalizações disponíveis.

3. O usuário confirma as alterações desejadas.

4. O servidor valida autoria, formatos, disponibilidade e elegibilidade.

5. O sistema persiste a alteração e reapresenta o perfil confirmado.

Fluxos alternativos e exceções

A1 — Nome indisponível, formato inválido ou intervalo de alteração não atendido: a edição é recusada com a causa correspondente.

A2 — Título, moldura ou cosmético não desbloqueado: o sistema mantém a seleção anterior.

A3 — Imagem incompatível ou falha no envio: o perfil mantém a imagem confirmada e permite nova tentativa.

Pós-condição de sucesso: Perfil e preferências autorizadas persistidos.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/api/upload/route.ts, src/app/settings/page.tsx, src/services/profile-customization.service.ts.

Regras: R01, R02, R03, R28, R93.

UC004 Consultar Trilhas de Estudo

Objetivo: Escolher uma linguagem e consultar o percurso, as unidades e seu progresso.

Atores: Usuário

Pré-condições: Sessão válida para consultar o progresso pessoal; currículo disponível na versão instalada.

Fluxo principal

1. O usuário abre Trilhas e seleciona linguagem, caminho e modo de visualização.

2. O sistema carrega o currículo e o progresso pessoal.

3. A interface apresenta unidades, lições, exercícios concluídos e requisitos.

4. O usuário seleciona uma lição disponível para continuar ou revisar.

Fluxos alternativos e exceções

A1 — Troca de linguagem ou caminho: a interface carrega o percurso escolhido e seu progresso independente.

A2 — Lição bloqueada por pré-requisito: o sistema identifica a etapa necessária e impede acesso indevido.

A3 — Caminho concluído: as lições continuam disponíveis para revisão, sem repetir o crédito de conclusão.

Pós-condição de sucesso: Percurso e progresso apresentados para a seleção atual.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/trails/TrailsContent.tsx, src/lib/learning/progress.ts.

Regras: R08, R09, R10.

UC005 Realizar Lição

Objetivo: Estudar o conteúdo de uma lição e avançar para suas atividades.

Atores: Usuário

Pré-condições: Sessão válida; lição existente e acessível no caminho selecionado.

Fluxo principal

1. O usuário abre uma lição disponível.

2. O sistema verifica a lição e o progresso correspondente.

3. A interface apresenta conceito, exemplo e orientação da etapa.

4. O usuário lê o conteúdo e avança às atividades de UC006.

5. O sistema permite retornar ao percurso e consultar o progresso confirmado.

Fluxos alternativos e exceções

A1 — Retomar uma lição: o sistema usa o progresso persistido e permite continuar ou revisar.

A2 — Sair antes de concluir: os registros já confirmados permanecem, enquanto a sessão local de estudo é encerrada.

A3 — Lição inexistente ou bloqueada: o sistema orienta o retorno ao caminho e informa a restrição.

Pós-condição de sucesso: Conteúdo estudado e etapas confirmadas disponíveis para continuidade.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/lib/learning/lesson-progress.ts.

Regras: R11, R12, R20.

UC006 Resolver exercício

Objetivo: Resolver a atividade, verificar a resposta e confirmar o resultado da aprendizagem.

Atores: Usuário

Pré-condições: Sessão válida; atividade disponível e vinculada à lição ou ao exercício selecionado.

Fluxo principal

1. O usuário abre a atividade e consulta enunciado, opções ou editor.

2. O usuário informa uma resposta compatível com o tipo da atividade.

3. O servidor valida o contexto e solicita UC026 para avaliar a resposta.

4. A interface apresenta acerto, erro ou orientação aplicável.

5. Na submissão elegível, o sistema registra a tentativa e confirma a conclusão e a recompensa permitida.

Fluxos alternativos e exceções

A1 — Executar uma prévia: o código é testado, mas a prévia não conclui a atividade nem concede XP.

A2 — Resposta incorreta: o sistema apresenta o retorno previsto para o tipo de atividade e permite nova tentativa conforme a regra de vidas e progresso.

A3 — Solicitar ajuda: realiza-se UC012 e o estudante retorna à atividade com o modo de assistência registrado.

A4 — Repetir atividade já concluída: o sistema permite a revisão sem duplicar a recompensa de primeira conclusão.

Pós-condição de sucesso: Tentativa avaliada e submissão elegível confirmada sem duplicação de recompensa.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/api/lessons/[lessonId]/attempt/route.ts, src/lib/exercises/repository.ts, src/lib/learning/assessment.ts, src/lib/learning/lesson-progress.ts, src/services/daily-quiz.service.ts.

Regras: R12, R13, R14, R15, R16, R17, R18, R19, R25, R89.

UC007 Emitir Classificação e Desempenho

Objetivo: Apurar e apresentar progresso, XP, nível, classificação e reconhecimento.

Atores: Usuário, com atualização automática pelo sistema

Pré-condições: Existir resultado validado ou consulta de desempenho autorizada; usar o período e a linguagem selecionados.

Fluxo principal

1. O sistema recebe um resultado confirmado ou uma consulta de desempenho.

2. O sistema considera os registros válidos, a linguagem e o período aplicáveis.

3. O sistema apura XP, nível, progresso e critérios de reconhecimento.

4. A interface apresenta a classificação e o desempenho correspondentes.

5. Quando os critérios forem atendidos, o sistema disponibiliza o reconhecimento elegível.

Fluxos alternativos e exceções

A1 — Filtro por linguagem ou período: a classificação é recalculada para o filtro escolhido.

A2 — Reconhecimento ainda inelegível: o sistema mostra os requisitos faltantes e não emite uma credencial indevida.

A3 — Credencial já emitida: o registro existente é reutilizado; verificações públicas e PDF seguem suas condições de acesso.

Pós-condição de sucesso: Desempenho, classificação e reconhecimento apresentados com base nos registros válidos.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/lib/credentials/pdf.ts, src/lib/credentials/repository.ts, src/lib/learning/rewards.ts.

Regras: R22, R23, R24, R25, R26, R27, R28.

UC008 Realizar Duelo

Objetivo: Participar de um confronto de programação, da formação da partida ao resultado.

Atores: Usuário e adversário

Pré-condições: Sessão válida; linguagem e participantes elegíveis; respeitar os limites da fila e as regras contra abandono.

Fluxo principal

1. O usuário escolhe a linguagem e solicita a fila ou cria um desafio.

2. O sistema valida elegibilidade e forma a partida quando houver adversário compatível.

3. Os participantes recebem o problema e as condições da disputa.

4. Cada participante trabalha em sua solução e usa UC010 para enviar.

5. O sistema aplica a política de encerramento e apresenta o resultado.

Fluxos alternativos e exceções

A1 — Fila sem adversário: o jogador aguarda ou cancela a busca antes da formação da partida.

A2 — Convite recusado ou expirado: o sistema encerra o pedido conforme sua política, sem simular aceitação.

A3 — Prazo, empate, abandono ou revisão: o serviço aplica a política de resolução e registra o motivo do encerramento.

A4 — Solução do participante: o envio ocorre por UC010, com execução e testes de UC027.

Pós-condição de sucesso: Partida formada e encerrada conforme sua política, com resultado registrado.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/lib/duels/resolution-policy.ts, src/services/duel.service.ts.

Regras: R61, R62, R64, R65, R66, R67, R68, R69, R70, R71.

UC010 Enviar Solução de Duelo

Objetivo: Enviar uma solução elegível ao duelo e registrar sua avaliação técnica.

Atores: Participante do duelo

Pré-condições: Sessão válida; usuário participante; duelo existente e em fase que aceite submissão.

Fluxo principal

1. O participante abre o duelo e informa o código na linguagem permitida.

2. O participante solicita o envio da solução.

3. O servidor confirma participação, fase, prazo e limites da submissão.

4. O sistema solicita UC027 e registra os resultados de execução e testes.

5. O sistema vincula o envio ao duelo e apresenta a situação confirmada.

Fluxos alternativos e exceções

A1 — Apenas executar: retorna a saída de teste sem confundir a prévia com o envio definitivo.

A2 — Código inválido, teste reprovado ou execução indisponível: o sistema registra o resultado permitido e informa o erro; não declara aprovação fictícia.

A3 — Usuário fora da partida ou prazo encerrado: o envio é recusado antes da alteração do resultado.

Pós-condição de sucesso: Solução vinculada ao participante e ao duelo, com resultado técnico registrado.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/lib/duels/judge.ts.

Regras: R63, R64.

UC012 Consultar Ajuda de Exercício

Objetivo: Consultar uma orientação para avançar na resolução de uma atividade.

Atores: Usuário

Pré-condições: Atividade acessível; usuário autenticado; modalidade de ajuda permitida.

Fluxo principal

1. Durante UC006, o usuário solicita a ajuda disponível.

2. O sistema verifica o contexto da atividade e o modo de assistência permitido.

3. O sistema registra a assistência aplicável e retorna a orientação.

4. O usuário retoma a resolução da atividade.

Fluxos alternativos e exceções

A1 — Ajuda sem revelar a resposta: o sistema apresenta a dica correspondente e mantém a atividade aberta.

A2 — Assistência com efeito na recompensa: o modo utilizado é registrado e considerado pela política da atividade.

A3 — Ajuda indisponível ou não autorizada: a atividade permanece acessível sem conceder assistência não prevista.

Pós-condição de sucesso: Orientação apresentada e modo de assistência registrado quando aplicável.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/lib/exercises/repository.ts.

Regras: R18, R19.

UC013 Gerenciar Publicações

Objetivo: Criar, consultar, editar ou excluir uma publicação própria.

Atores: Usuário

Pré-condições: Sessão válida para alterações; autorização sobre o conteúdo; atender aos limites de texto, mídia e audiência.

Fluxo principal

1. O usuário abre o formulário de publicação ou uma publicação própria.

2. O usuário informa ou altera o texto e os recursos compatíveis.

3. O servidor verifica autorização, limites e referências dos anexos.

4. O sistema grava a publicação ou a alteração solicitada.

5. A interface apresenta o conteúdo confirmado no feed ou no perfil.

Fluxos alternativos e exceções

A1 — Editar ou excluir: o sistema verifica a autoria ou a permissão administrativa antes de confirmar a alteração.

A2 — Incluir código, imagem, enquete ou outro recurso compatível: o conteúdo segue as validações específicas do recurso.

A3 — Dados inválidos ou falha na persistência: o conteúdo não é tratado como publicado e o formulário permite correção.

Pós-condição de sucesso: Publicação criada ou mantida com a autorização correspondente.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/post.service.ts.

Regras: R02, R07, R29, R30.

UC014 Registrar Interação

Objetivo: Registrar uma interação permitida com o conteúdo exibido.

Atores: Usuário

Pré-condições: Sessão válida para persistir a ação; publicação acessível; respeitar bloqueios e restrições de audiência.

Fluxo principal

1. O usuário consulta uma publicação e escolhe uma interação disponível.

2. O sistema verifica sessão, acesso ao conteúdo e validade da ação.

3. O serviço correspondente registra a reação, resposta, voto ou item salvo.

4. A interface atualiza o estado da interação a partir do resultado confirmado.

Fluxos alternativos e exceções

A1 — Reagir novamente à mesma publicação: o serviço altera ou remove a reação conforme a ação, sem duplicação indevida.

A2 — Responder a uma conversa: a resposta é vinculada à publicação e, quando aplicável, à resposta anterior.

A3 — Salvar ou retirar dos itens salvos: somente a coleção pessoal do titular é modificada.

A4 — Votar em enquete: opções inválidas, voto incompatível ou enquete encerrada impedem a alteração.

Pós-condição de sucesso: Interação confirmada no registro e refletida na interface.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/api/posts/[id]/answer/route.ts, src/app/api/posts/[id]/poll/route.ts, src/app/bookmarks/page.tsx, src/services/reaction.service.ts.

Regras: R07, R29, R31, R32, R33, R34, R91.

UC015 Gerenciar Eventos

Objetivo: Consultar e administrar um evento, seus desafios e sua situação.

Atores: Administrador ou responsável autorizado pelo evento

Pré-condições: Sessão válida; autorização administrativa ou vínculo com o evento; evento existente.

Fluxo principal

1. O responsável abre a administração de eventos.

2. O sistema verifica a autorização e lista os eventos acessíveis.

3. O responsável consulta ou altera informações, desafios e situação do evento.

4. O sistema valida a operação e grava os dados permitidos.

5. A interface confirma a atualização e disponibiliza os avisos elegíveis de UC021.

Fluxos alternativos e exceções

A1 — Atualizar os desafios: a ordenação, os enunciados e os dados de avaliação são validados antes da gravação.

A2 — Participação do usuário: inscrição, lotação, repetição e cancelamento seguem o serviço de eventos e a janela permitida.

A3 — Evento encerrado ou usuário sem autorização: o sistema mantém o registro e informa a restrição.

Pós-condição de sucesso: Evento administrado com os vínculos e limites preservados.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/event-challenge.service.ts, src/services/event.service.ts.

Regras: R81, R82, R83, R84, R85, R86, R87.

UC016 Avaliar Duelo

Objetivo: Registrar um parecer humano autorizado sobre um duelo.

Atores: Avaliador

Pré-condições: Sessão válida; credenciamento e autorização vigentes; duelo apto à avaliação; ausência de conflito de participação.

Fluxo principal

1. O avaliador abre um duelo elegível para revisão.

2. O sistema verifica credenciamento, autorização e impedimentos.

3. O avaliador consulta as soluções e os critérios e informa seu parecer.

4. O servidor valida notas, justificativa e situação da avaliação.

5. O sistema registra o parecer e aplica os efeitos previstos à resolução do duelo.

Fluxos alternativos e exceções

A1 — Avaliador participante ou impedido: o sistema recusa o parecer para preservar a imparcialidade.

A2 — Nota, justificativa ou critérios inválidos: o formulário requer correção antes da persistência.

A3 — Parecer já existente ou duelo fora da fase elegível: o fluxo segue a política de avaliação e não duplica o resultado.

Pós-condição de sucesso: Parecer autorizado registrado e considerado pela política de resolução.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/evaluator.service.ts.

Regras: R73, R74, R75.

UC017 Gerenciar Empresas

Objetivo: Manter o cadastro da empresa e o vínculo de seu responsável.

Atores: Recrutador e administrador, conforme autorização do recurso

Pré-condições: Sessão válida; papel e vínculo exigidos pela operação.

Fluxo principal

1. O gestor abre a área de empresas e informa ou seleciona a empresa.

2. O sistema confere seu papel e o vínculo de responsabilidade.

3. O gestor informa ou atualiza os dados cadastrais permitidos.

4. O sistema valida e grava o cadastro da empresa.

5. A interface confirma a empresa e permite acessar suas oportunidades autorizadas.

Fluxos alternativos e exceções

A1 — Atualizar empresa existente: o sistema confere o vínculo do gestor antes de alterar os dados.

A2 — Empresa inexistente, identificador em conflito ou dados inválidos: a ação é recusada com indicação da causa.

A3 — Acesso a outra empresa sem vínculo: o sistema impede a operação, mesmo quando a interface apresentou o recurso.

Pós-condição de sucesso: Empresa cadastrada ou atualizada e vinculada ao responsável autorizado.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/job.service.ts.

Regras: R04, R79.

UC018 Gerenciar Vagas

Objetivo: Cadastrar e manter uma vaga vinculada à empresa correta.

Atores: Recrutador ou administrador autorizado

Pré-condições: Sessão válida; autorização sobre a empresa; campos da oportunidade e etapas válidos.

Fluxo principal

1. O gestor seleciona uma empresa autorizada e abre o cadastro ou a edição da vaga.

2. O gestor informa título, descrição, tecnologias, modalidade e etapas.

3. O servidor verifica o vínculo com a empresa e valida a oportunidade.

4. O sistema grava a vaga e suas etapas de recrutamento.

5. A interface confirma a oportunidade e os avisos elegíveis seguem UC021.

Fluxos alternativos e exceções

A1 — Alterar, encerrar ou filtrar uma vaga: o sistema aplica a autorização e a situação atual do registro.

A2 — Empresa sem autorização: a vaga não é criada ou alterada.

A3 — Etapas ou dados inválidos: a operação retorna para correção sem confirmar publicação incompleta.

Pós-condição de sucesso: Vaga e etapas persistidas na empresa autorizada.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/job.service.ts.

Regras: R04, R79.

UC019 Gerenciar Usuários

Objetivo: Consultar e administrar usuários e permissões dentro dos limites de acesso.

Atores: Administrador

Pré-condições: Sessão válida; papel administrativo confirmado pelo servidor.

Fluxo principal

1. O administrador abre a área restrita de usuários.

2. O servidor verifica o papel administrativo e carrega os registros autorizados.

3. O administrador consulta um usuário e solicita a operação permitida.

4. O sistema valida a alteração e registra os dados e a auditoria aplicável.

5. A interface apresenta o resultado confirmado.

Fluxos alternativos e exceções

A1 — Solicitação de credenciamento: o administrador analisa o pedido e aprova ou rejeita com a justificativa correspondente.

A2 — Filtro de usuários ou auditoria: o sistema retorna somente os registros e campos autorizados.

A3 — Tentativa sem permissão: o recurso retorna recusa de acesso e não altera papéis ou dados.

Pós-condição de sucesso: Operação administrativa autorizada registrada.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/admin.service.ts.

Regras: R02, R03.

UC021 Notificar Usuário

Objetivo: Registrar e disponibilizar o aviso produzido por uma operação relevante.

Atores: Sistema, com Usuário como destinatário

Pré-condições: Ocorrer fato notificável; existir destinatário válido; respeitar os critérios da notificação.

Fluxo principal

1. Uma operação de origem identifica um fato notificável e seu destinatário.

2. O sistema valida a categoria, o destinatário e os dados do aviso.

3. O serviço registra a notificação.

4. A interface do destinatário consulta ou recebe a atualização dos avisos.

5. O usuário pode filtrar, consultar e marcar suas notificações conforme a ação permitida.

Fluxos alternativos e exceções

A1 — Consultar por categoria ou marcar leitura: somente as notificações do titular são afetadas.

A2 — Nenhum aviso para o filtro: a interface apresenta o estado vazio.

A3 — Falha na atualização automática: a interface mantém os dados confirmados e permite reconsulta, sem presumir que o destinatário leu o aviso.

Pós-condição de sucesso: Aviso registrado para o destinatário e disponível para consulta.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/notification.service.ts.

Regras: R35, R38.

UC022 Registrar Candidatura

Objetivo: Registrar uma candidatura única a uma vaga aberta.

Atores: Usuário candidato

Pré-condições: Sessão válida; vaga existente e aberta; usuário ainda não inscrito na mesma oportunidade.

Fluxo principal

1. O usuário consulta uma vaga e solicita a candidatura.

2. O servidor verifica a sessão, a existência da vaga e sua situação aberta.

3. O sistema verifica se já existe candidatura do mesmo usuário à vaga.

4. O sistema cria a candidatura com sua situação inicial e os vínculos corretos.

5. A interface confirma a inscrição e disponibiliza seu acompanhamento.

Fluxos alternativos e exceções

A1 — Candidatura já existente: o sistema informa a inscrição anterior e não cria outra.

A2 — Vaga encerrada ou removida: o envio é recusado antes da gravação.

A3 — Falha de rede: o usuário consulta a situação confirmada antes de repetir o envio.

Pós-condição de sucesso: Candidatura única registrada na vaga aberta.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/job.service.ts.

Regras: R76, R77, R78.

UC023 Criar evento

Objetivo: Criar um evento ou hackathon com suas informações obrigatórias.

Atores: Usuário com permissão de criação no fluxo

Pré-condições: Sessão válida; permissão de criação confirmada; título, período, formato e demais campos válidos.

Fluxo principal

1. O usuário autorizado abre o formulário de criação do evento.

2. O usuário informa os dados obrigatórios e o vínculo com empresa, quando aplicável.

3. O servidor confirma a permissão de criação e valida os dados.

4. O serviço cria o evento com sua situação inicial.

5. O sistema confirma o evento e encaminha os avisos elegíveis para UC021.

Fluxos alternativos e exceções

A1 — Evento associado a empresa: o sistema confere a autorização sobre a empresa vinculada.

A2 — Dados ou período inválidos: o formulário apresenta as correções necessárias.

A3 — Confirmação do cadastro: o evento passa à situação inicial e os avisos elegíveis seguem UC021.

Pós-condição de sucesso: Novo evento registrado com os dados válidos e situação inicial.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/event.service.ts.

Regras: R83, R84, R87.

UC024 Apresentar Pedido de Credenciamento

Objetivo: Apresentar motivação e conhecimentos para solicitar o credenciamento.

Atores: Usuário candidato a avaliador

Pré-condições: Sessão válida; cumprir os critérios de elegibilidade; não existir pedido pendente incompatível.

Fluxo principal

1. O usuário abre o pedido de credenciamento.

2. O sistema apresenta sua elegibilidade e os requisitos.

3. O candidato informa motivação e conhecimentos técnicos e envia o pedido.

4. O servidor verifica os requisitos e a inexistência de pedido pendente incompatível.

5. O sistema registra a solicitação para análise administrativa e apresenta sua situação.

Fluxos alternativos e exceções

A1 — Requisitos insuficientes: o sistema apresenta as condições ainda não atendidas e impede o pedido.

A2 — Solicitação pendente ou papel já credenciado: o sistema informa a situação existente.

A3 — Motivação ou tecnologias inválidas: o candidato corrige os campos; enviar o pedido não concede automaticamente o papel.

Pós-condição de sucesso: Pedido registrado para análise, sem concessão automática do papel.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/evaluator.service.ts.

Regras: R03, R72, R75.

UC025 Gerenciar Candidaturas

Objetivo: Consultar candidaturas e registrar sua movimentação no processo seletivo.

Atores: Recrutador ou administrador autorizado

Pré-condições: Sessão válida; vínculo autorizado com a empresa e a vaga; candidatura e etapa compatíveis.

Fluxo principal

1. O gestor abre o painel de candidaturas de uma vaga autorizada.

2. O sistema valida o vínculo com a empresa e apresenta os candidatos.

3. O gestor seleciona uma candidatura e informa etapa, situação ou feedback.

4. O servidor verifica a compatibilidade entre candidatura, vaga e etapa.

5. O sistema grava a movimentação, registra a auditoria aplicável e encaminha a notificação elegível.

Fluxos alternativos e exceções

A1 — Nova etapa, situação ou feedback: o sistema valida os vínculos, grava a mudança e registra a comunicação elegível.

A2 — Etapa pertencente a outra vaga: a movimentação é recusada.

A3 — Ausência de candidatos ou filtro sem resultado: o painel apresenta um estado vazio; nenhum registro é criado.

Pós-condição de sucesso: Movimentação da candidatura persistida com os vínculos corretos.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/services/job.service.ts.

Regras: R04, R79, R80.

UC026 Avaliar Resposta

Objetivo: Avaliar a resposta segundo o tipo de atividade e produzir um resultado consistente.

Atores: Sistema, acionado pela resposta do Usuário

Pré-condições: Resposta recebida em contexto de atividade válida; critérios de avaliação disponíveis no servidor.

Fluxo principal

1. O sistema recebe a resposta e identifica a atividade e seu tipo.

2. O avaliador do servidor valida formato e contexto.

3. O sistema compara a resposta aos critérios ou aciona UC027 quando houver execução de código.

4. O sistema produz um resultado de acerto, falha ou indisponibilidade.

5. Em uma submissão elegível, o sistema confirma tentativa, progresso e emissão de desempenho por UC007.

Fluxos alternativos e exceções

A1 — Atividade de código: a avaliação usa a execução e os testes de UC027 quando necessários.

A2 — Atividade objetiva ou interativa: a resposta é comparada ao formato e ao critério da atividade, sem exigir um executor externo.

A3 — Modo de prévia: o resultado retorna ao usuário sem creditar conclusão.

A4 — Primeira aprovação confirmada: os registros de progresso e desempenho elegíveis são atualizados, sem duplicar o crédito em repetições.

Pós-condição de sucesso: Resultado da avaliação produzido e efeitos elegíveis confirmados.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/api/lessons/[lessonId]/attempt/route.ts, src/lib/exercises/repository.ts, src/lib/learning/assessment.ts, src/lib/learning/lesson-progress.ts.

Regras: R12, R13, R14, R15, R16, R17, R18, R19, R25.

UC027 Executar Código e Testes

Objetivo: Executar código com limites e devolver saída, erros e resultado dos testes.

Atores: Executor de código, acionado pelo sistema

Pré-condições: Requisição autorizada; linguagem suportada; código e testes compatíveis; executor disponível.

Fluxo principal

1. O sistema prepara a requisição autorizada de código, linguagem e testes.

2. O executor recebe a requisição e aplica os limites de execução.

3. O executor executa o código e verifica os casos de teste.

4. O executor devolve saída, erros, duração e resultado técnico.

5. O serviço de origem interpreta o retorno segundo as regras da atividade ou do duelo.

Fluxos alternativos e exceções

A1 — Erro de sintaxe ou execução: a resposta diferencia falha do código de falha de infraestrutura.

A2 — Tempo ou recurso excedido: a execução termina segundo os limites do executor e retorna a condição correspondente.

A3 — Executor indisponível: o sistema informa a indisponibilidade; não converte a falha em teste aprovado.

Pós-condição de sucesso: Resultado técnico retornado ao fluxo de origem.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/api/lessons/[lessonId]/attempt/route.ts, src/lib/duels/judge.ts, src/lib/exercises/repository.ts.

Regras: R12, R13, R16, R17, R63, R64.

UC028 Gerenciar Denúncias

Objetivo: Analisar uma denúncia e registrar a providência de moderação.

Atores: Administrador

Pré-condições: Sessão válida; papel administrativo; denúncia existente e acessível à moderação.

Fluxo principal

1. O administrador abre a área de denúncias.

2. O sistema verifica o papel e apresenta os registros para análise.

3. O administrador consulta o conteúdo, o motivo e a situação da denúncia.

4. O administrador informa a providência autorizada ou a conclusão da análise.

5. O sistema registra a situação e confirma o resultado da moderação.

Fluxos alternativos e exceções

A1 — Denúncia procedente: o administrador confirma a medida autorizada e o sistema registra o resultado.

A2 — Denúncia improcedente ou já tratada: o registro segue a situação correspondente sem repetir a ação.

A3 — Recurso removido ou dados insuficientes: o sistema informa a condição e preserva os registros necessários à análise.

Pós-condição de sucesso: Análise e providência de moderação registradas.

Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/api/posts/[id]/report/route.ts, src/services/admin.service.ts.

Regras: R02, R03, R92.

UC029 Fazer Denúncia

Objetivo: Encaminhar uma denúncia sobre uma publicação acessível.

Atores: Usuário

Pré-condições: Sessão válida; conteúdo identificável; motivo e detalhes aceitos pelo formulário.

Fluxo principal

1. O usuário abre as opções da publicação e seleciona a denúncia.

2. O sistema apresenta os motivos e campos previstos.

3. O usuário informa o motivo e os detalhes e solicita o envio.

4. O servidor valida sessão, conteúdo e dados da denúncia.

5. O sistema registra o relato e confirma seu recebimento para moderação.

Fluxos alternativos e exceções

A1 — Motivo inválido ou detalhe obrigatório ausente: o sistema solicita correção.

A2 — Conteúdo indisponível ou denúncia incompatível: o envio é recusado com retorno adequado.

A3 — Denúncia aceita: o sistema confirma o recebimento; a denúncia não decide por si só a procedência ou a punição.

Pós-condição de sucesso: Denúncia recebida e disponível à moderação.

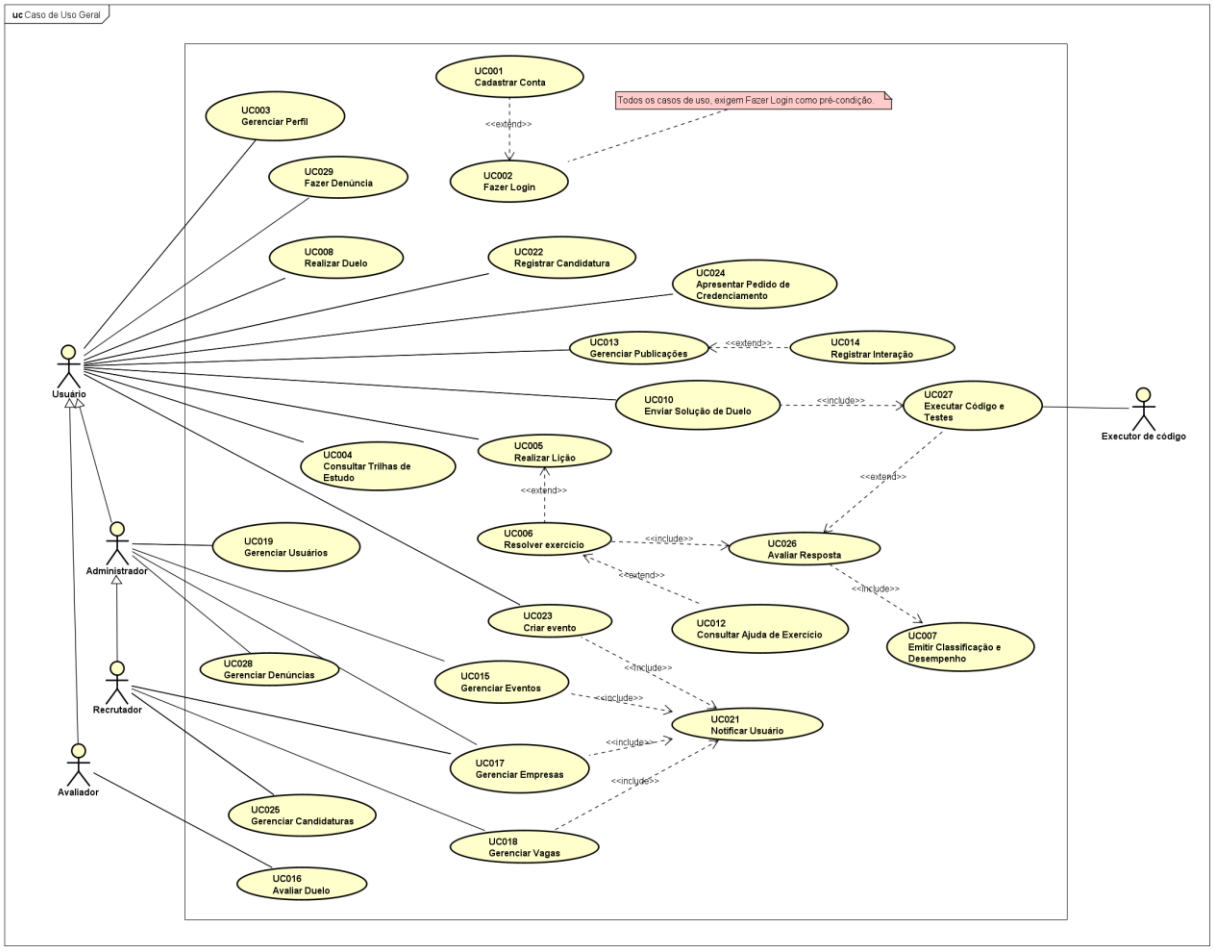
Garantia em caso de falha: A operação informa a causa da recusa ou da falha e mantém a última situação confirmada. Recompensas, permissões e resultados dependem da confirmação do servidor.

Implementação: src/app/api/posts/[id]/report/route.ts.

Regras: R02, R92.

Caso de Uso Geral

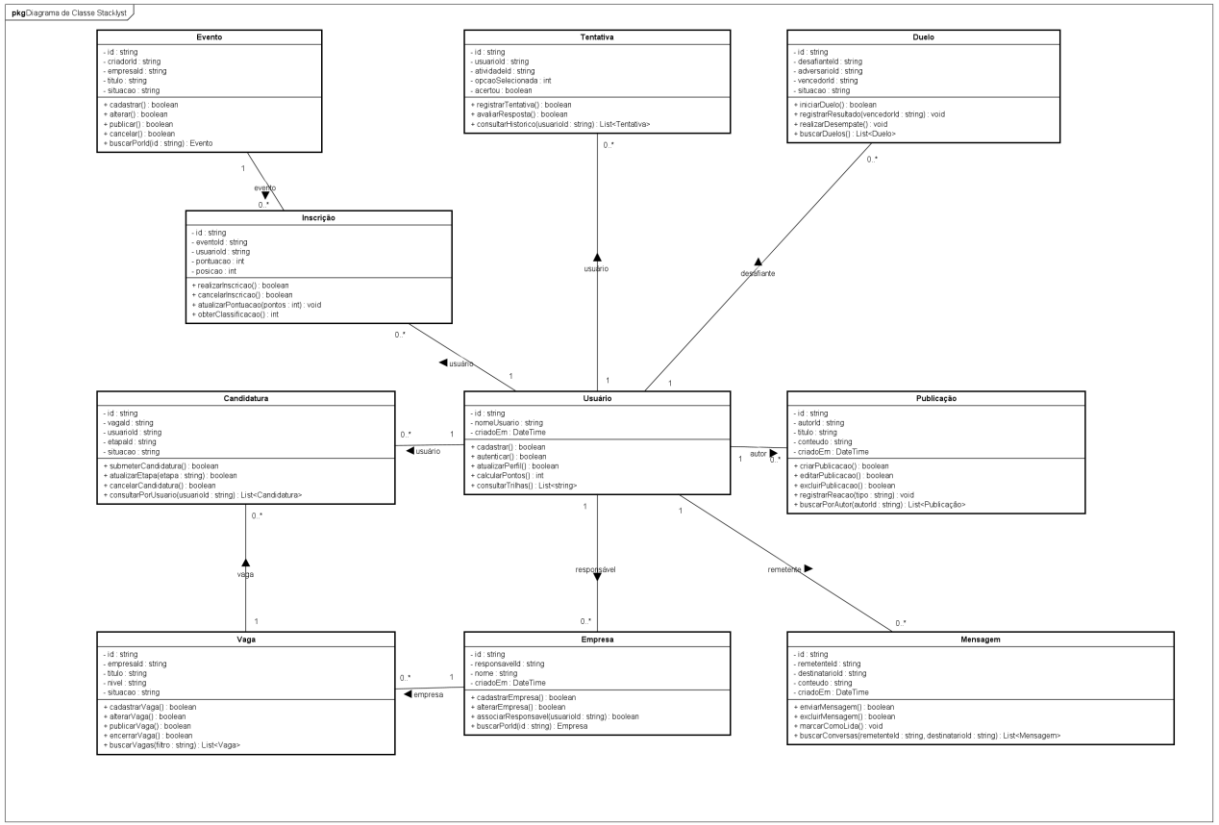
Fonte: Caso de Uso Geral.png fornecida pelo grupo.



Caso de Uso Geral — vista fornecida pelo grupo

Classes principais do Stacklyst

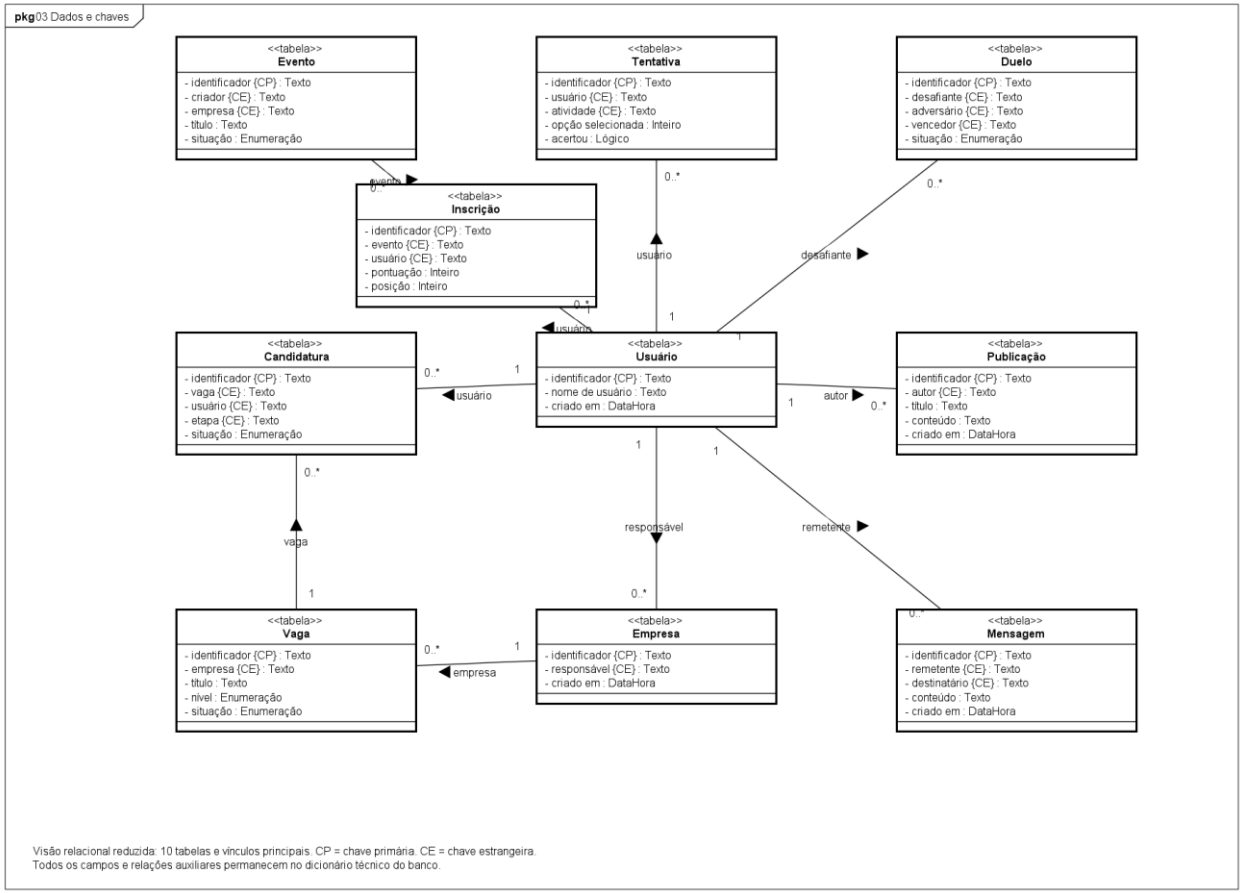
Fonte: Classes principais do Stacklyst.png fornecida pelo grupo.



Classes principais do Stacklyst — atributos e métodos

Visão relacional geral reduzida

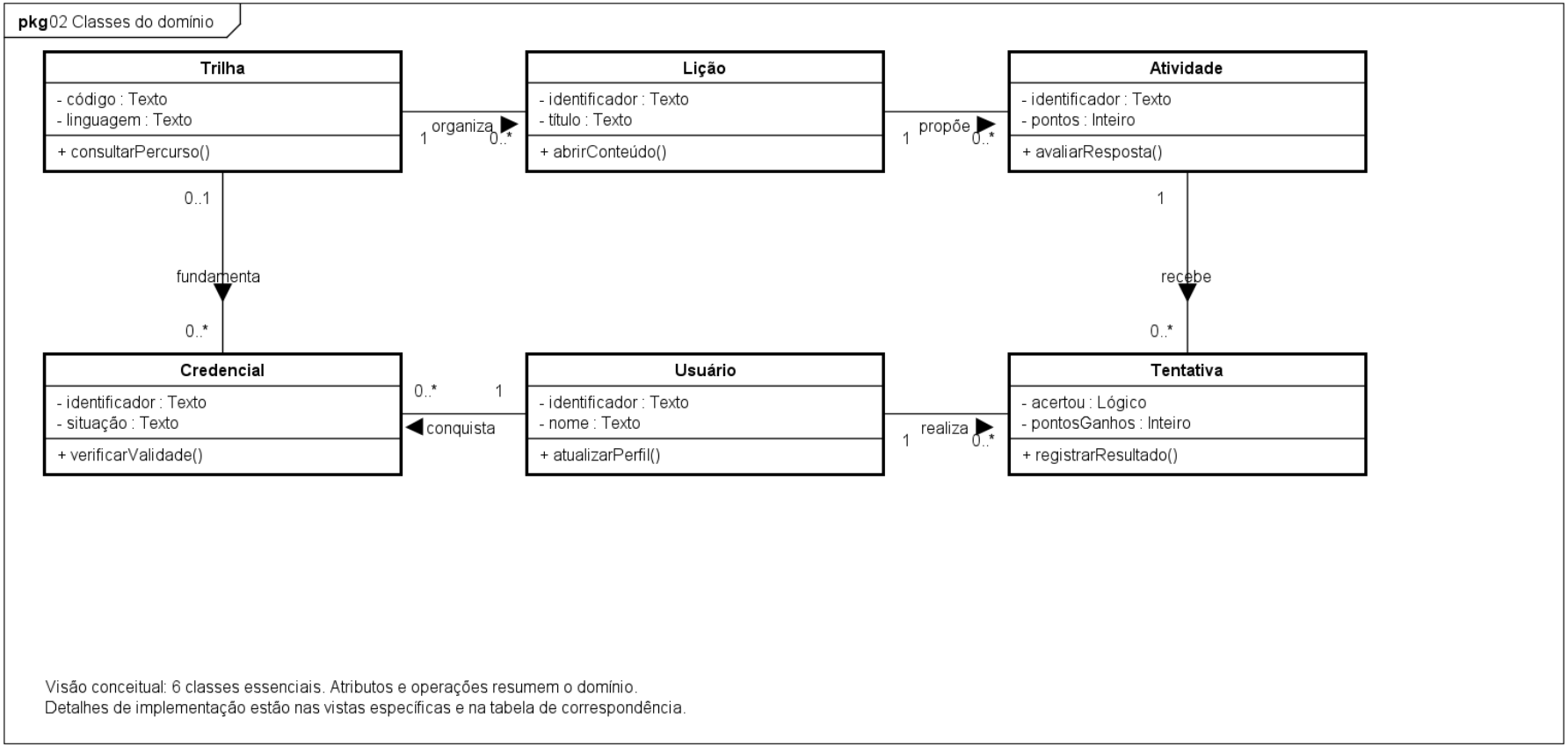
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-GERAL Visão relacional geral reduzida

Classes de aprendizagem

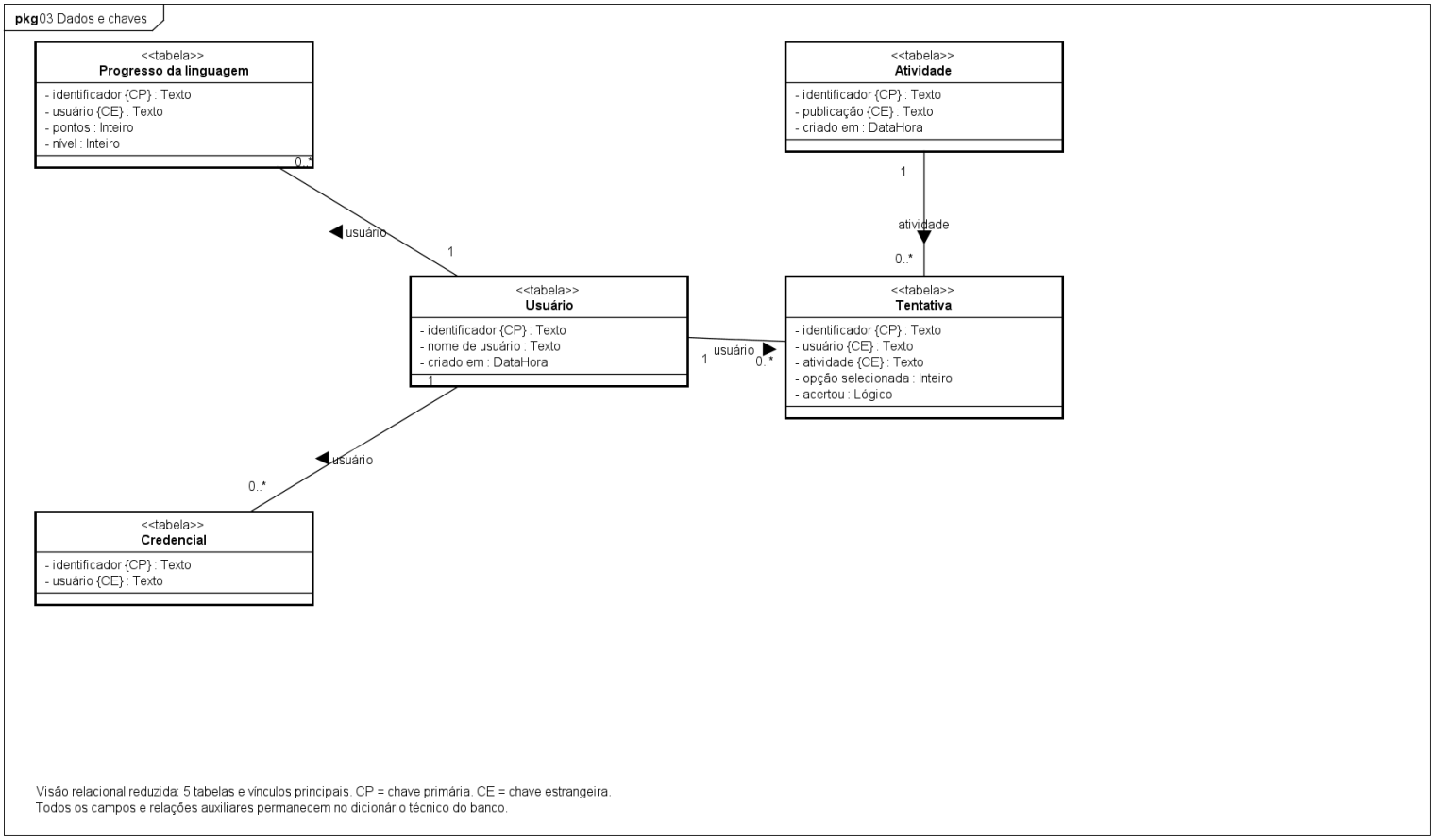
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-APRENDIZAGEM Classes de aprendizagem

Dados de aprendizagem

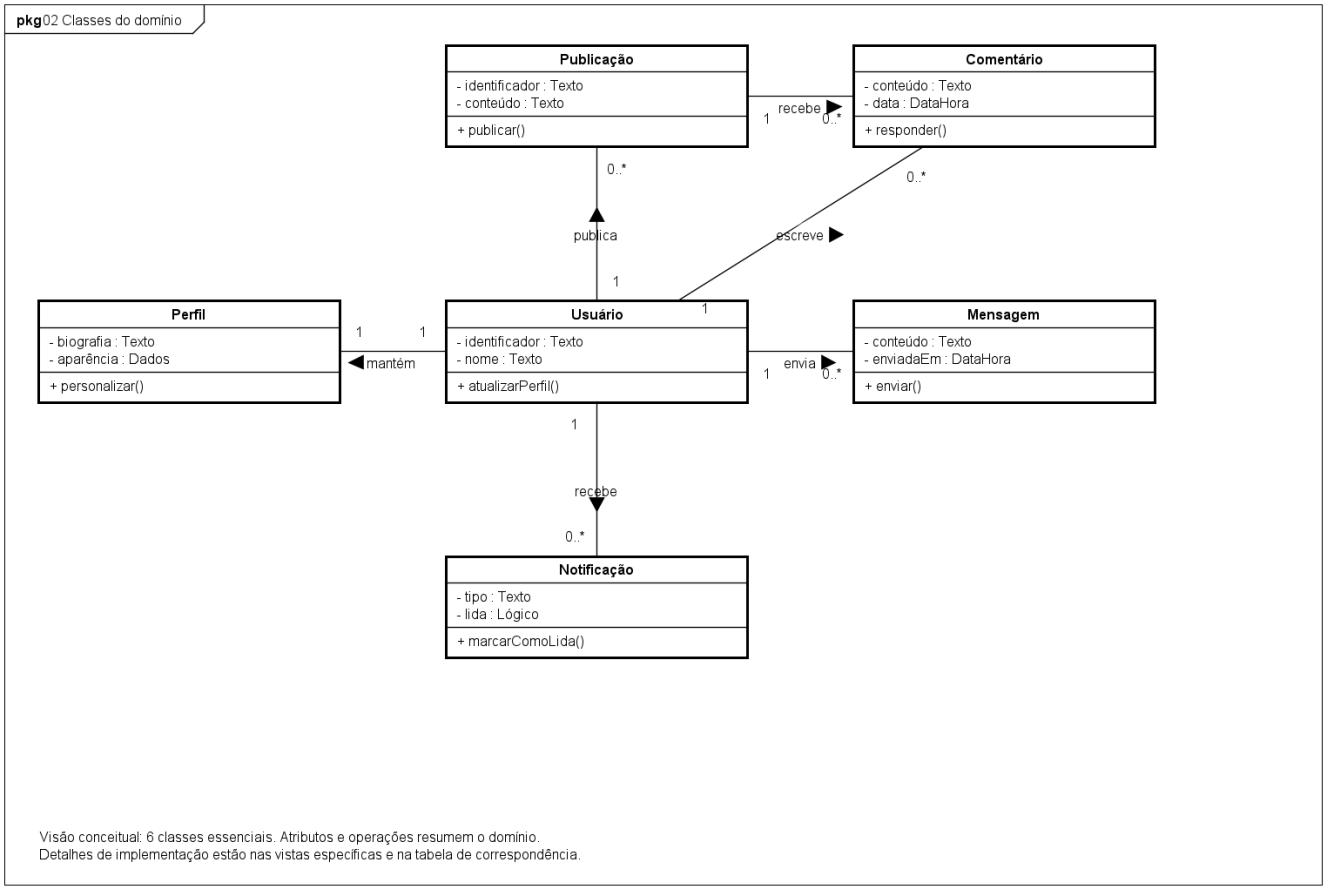
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-APRENDIZAGEM Dados de aprendizagem

Classes de comunidade

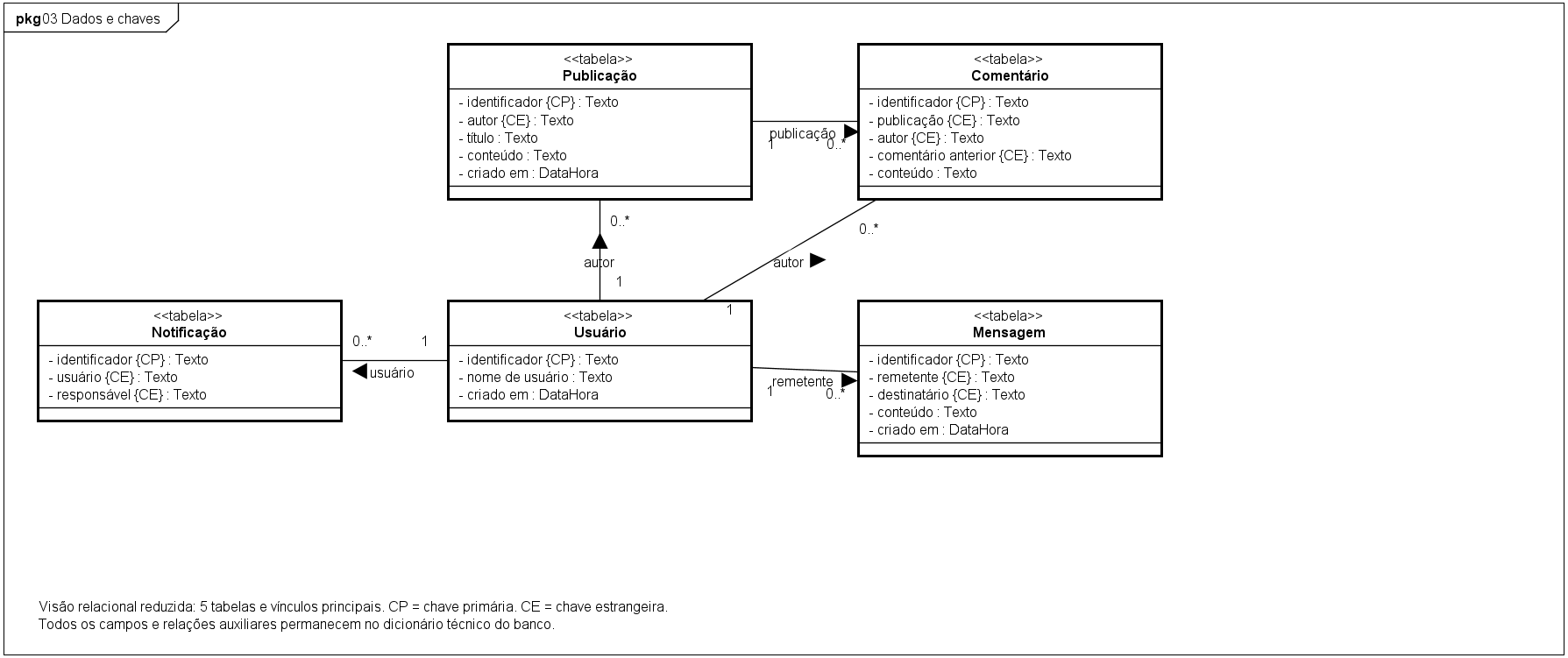
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-COMUNIDADE Classes de comunidade

Dados de comunidade

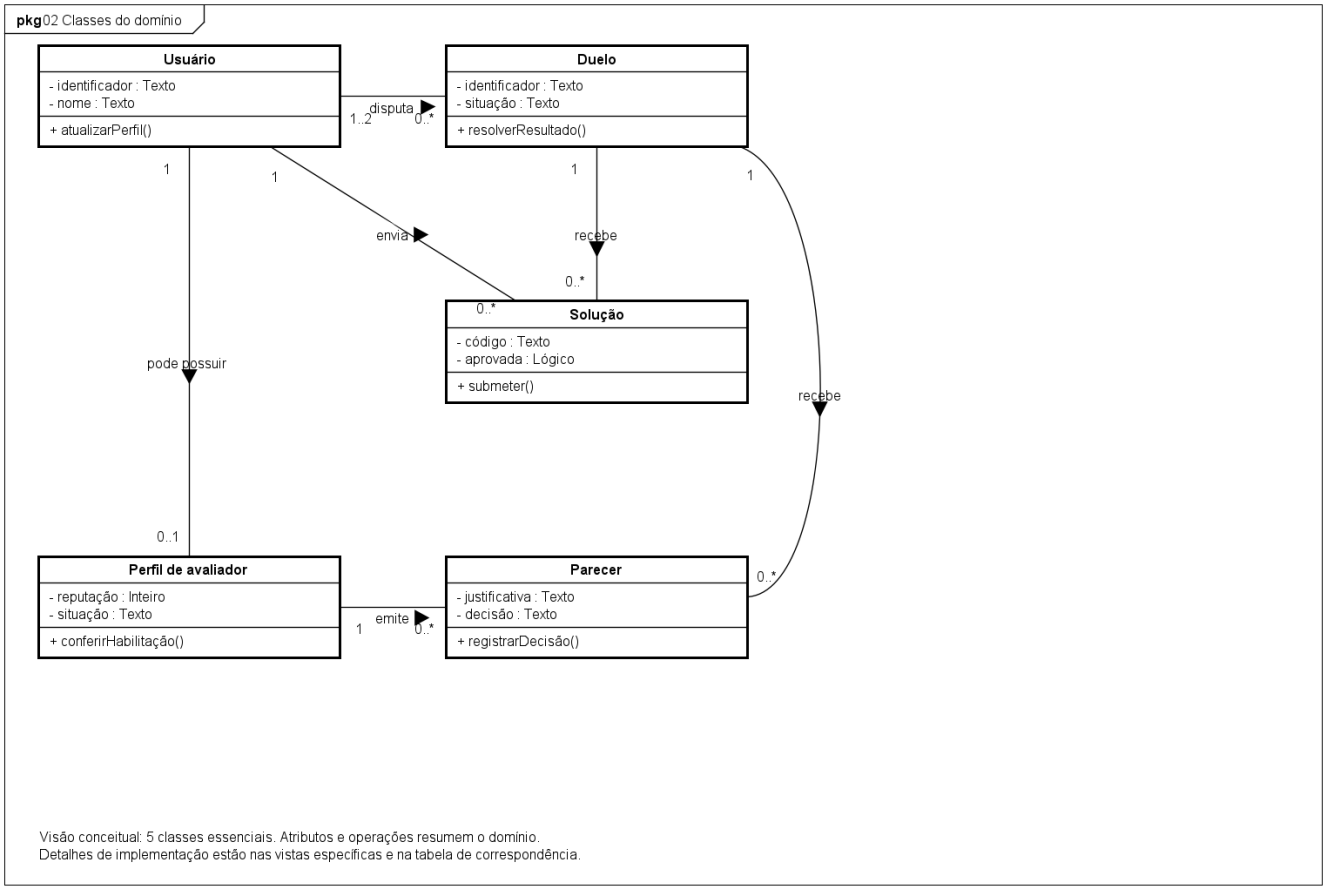
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-COMUNIDADE Dados de comunidade

Classes de duelos

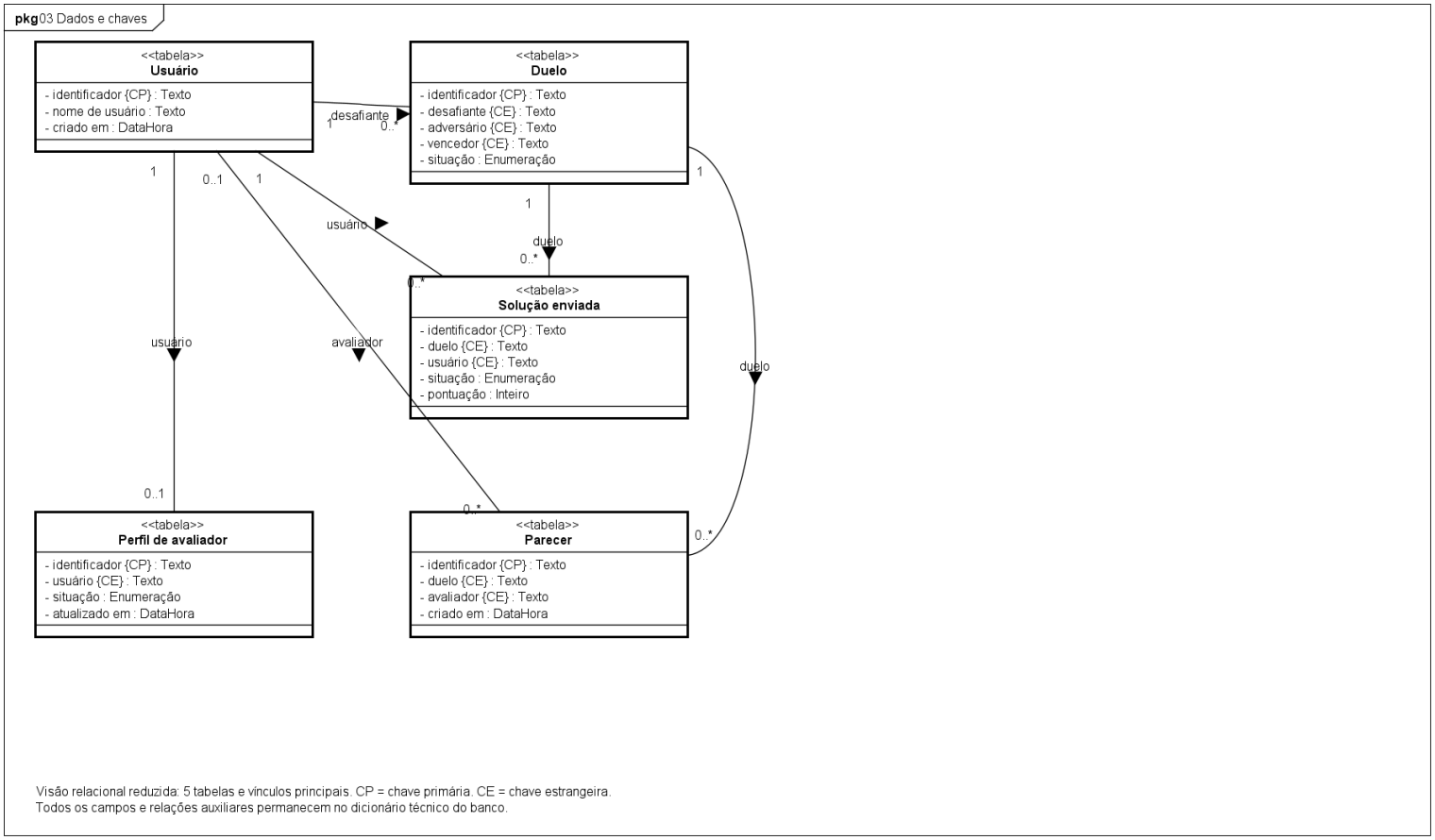
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-DUELOS Classes de duelos

Dados de duelos

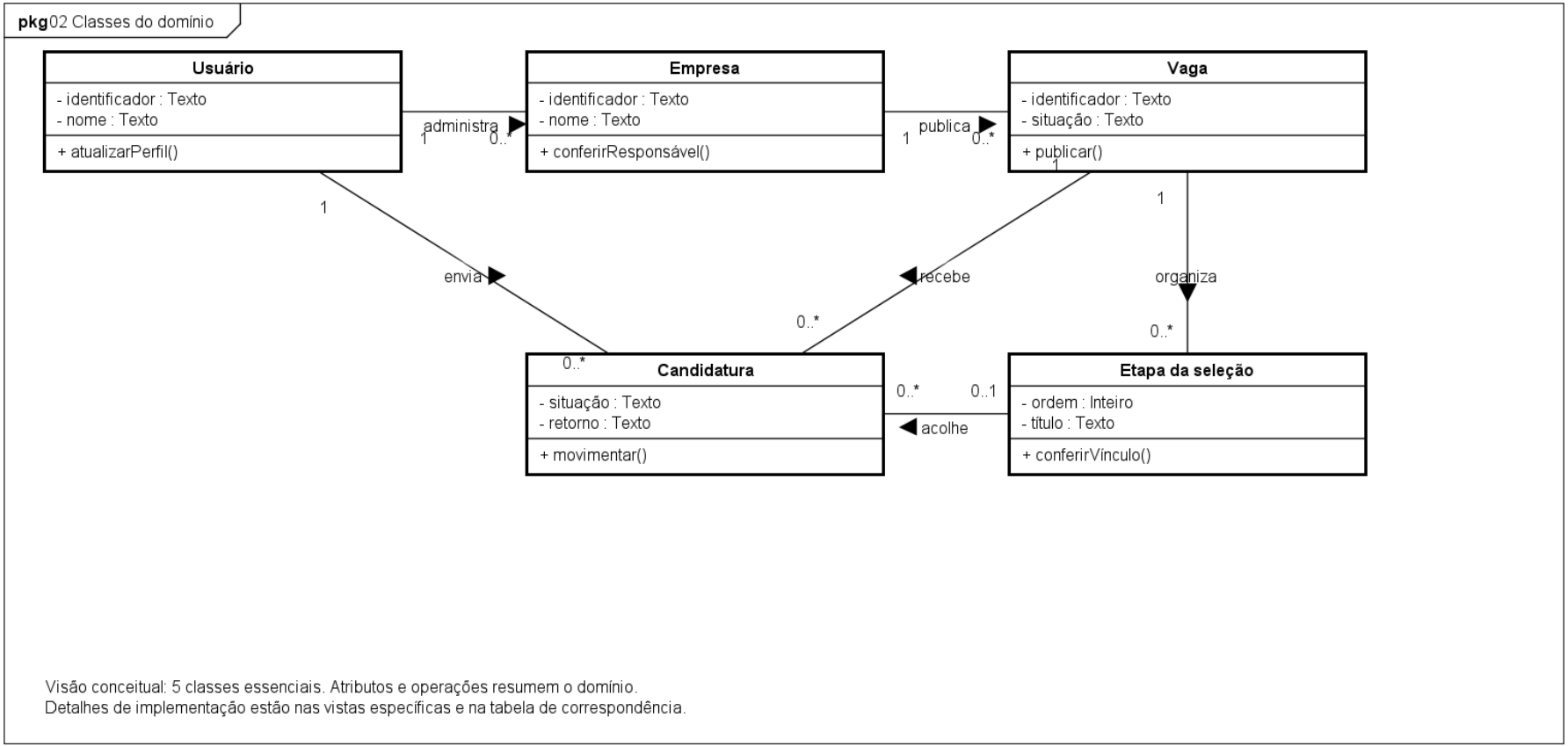
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-DUELOS Dados de duelos

Classes de recrutamento

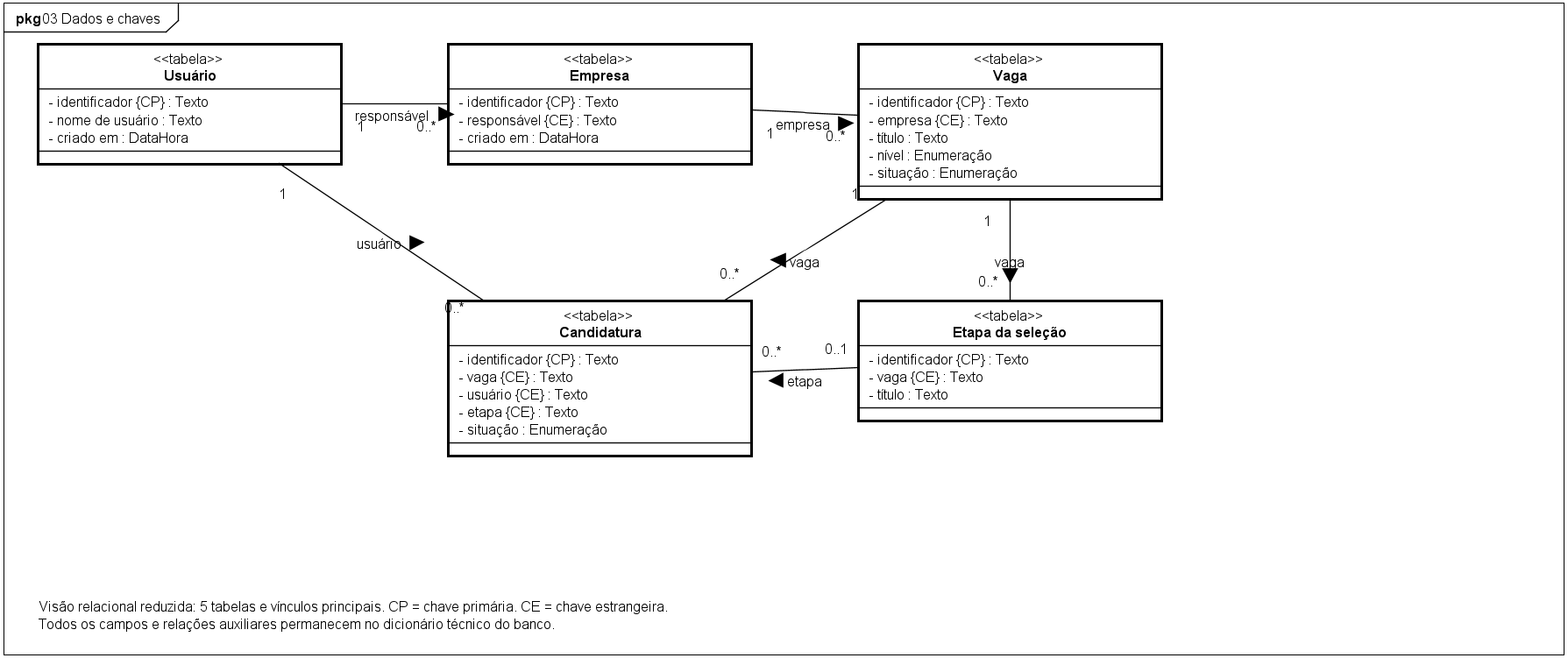
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-RECRUTAMENTO Classes de recrutamento

Dados de recrutamento

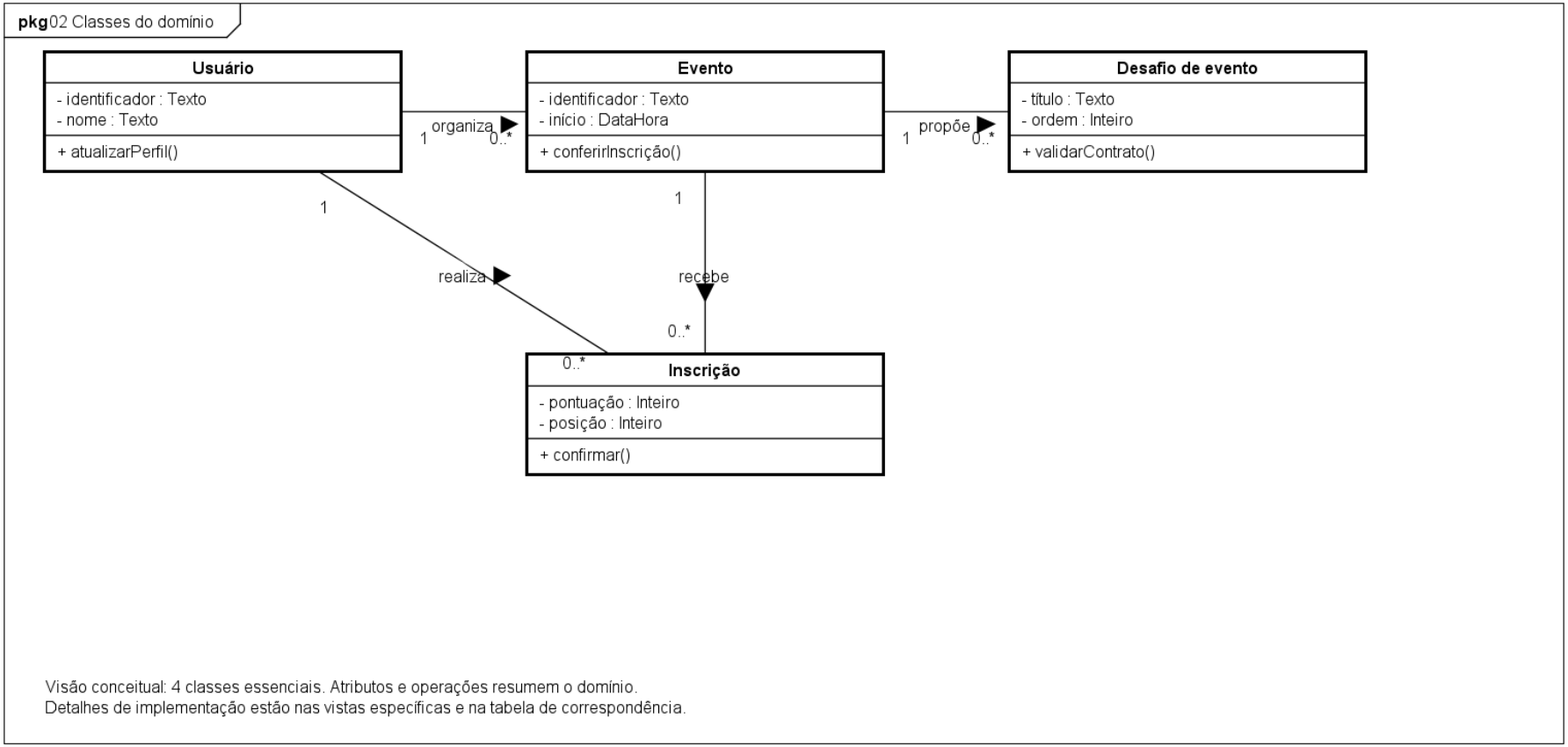
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-RECRUTAMENTO Dados de recrutamento

Classes de eventos

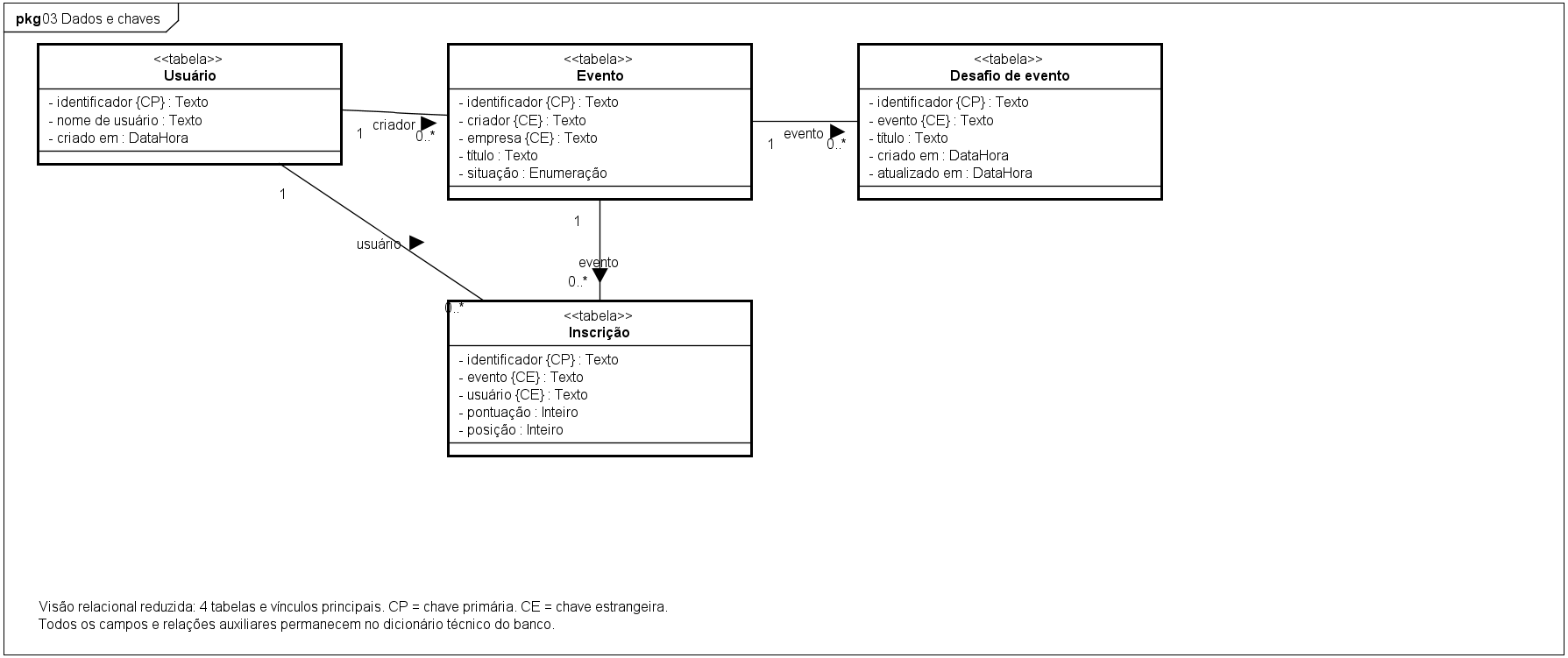
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-EVENTOS Classes de eventos

Dados de eventos

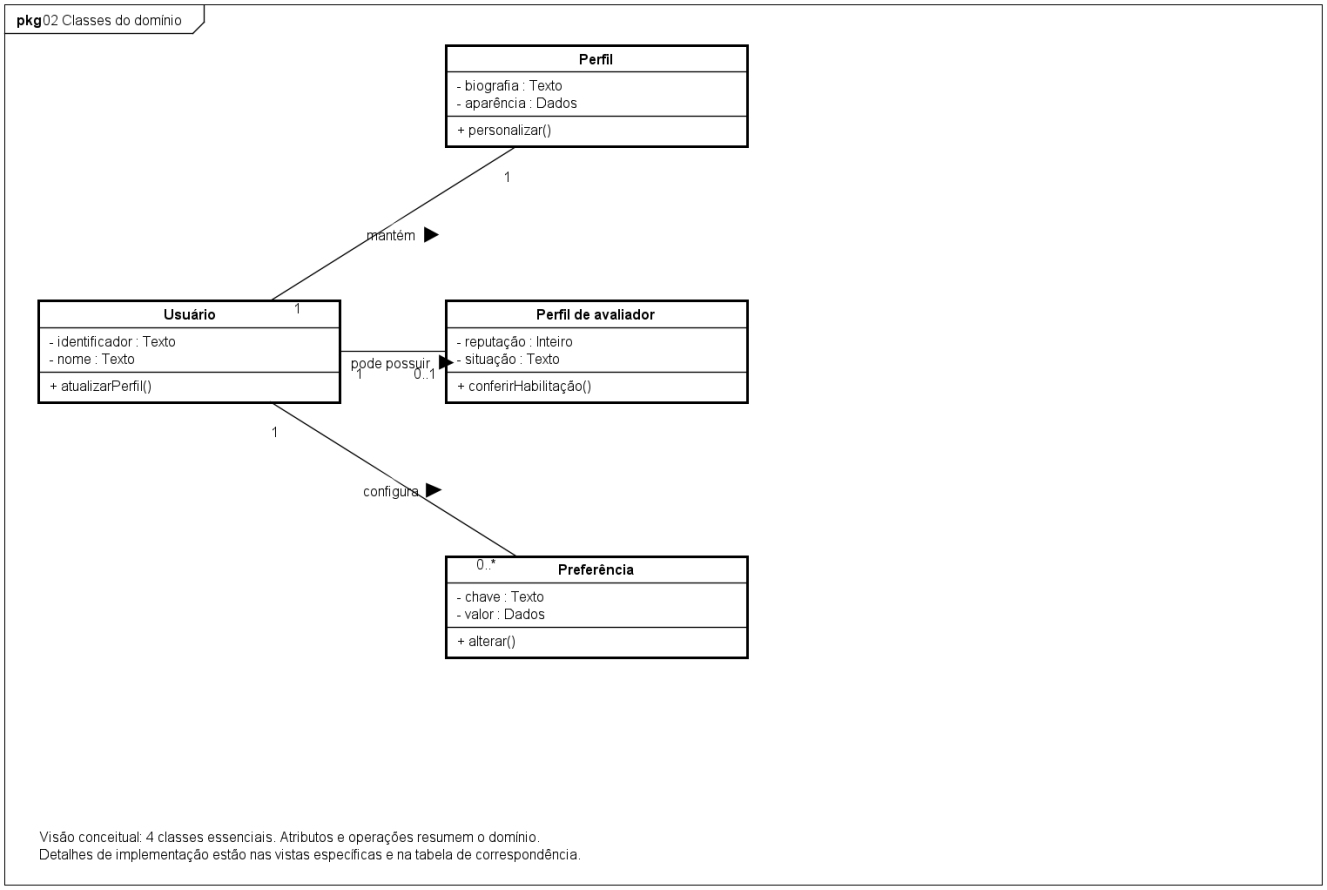
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-EVENTOS Dados de eventos

Classes de identidade

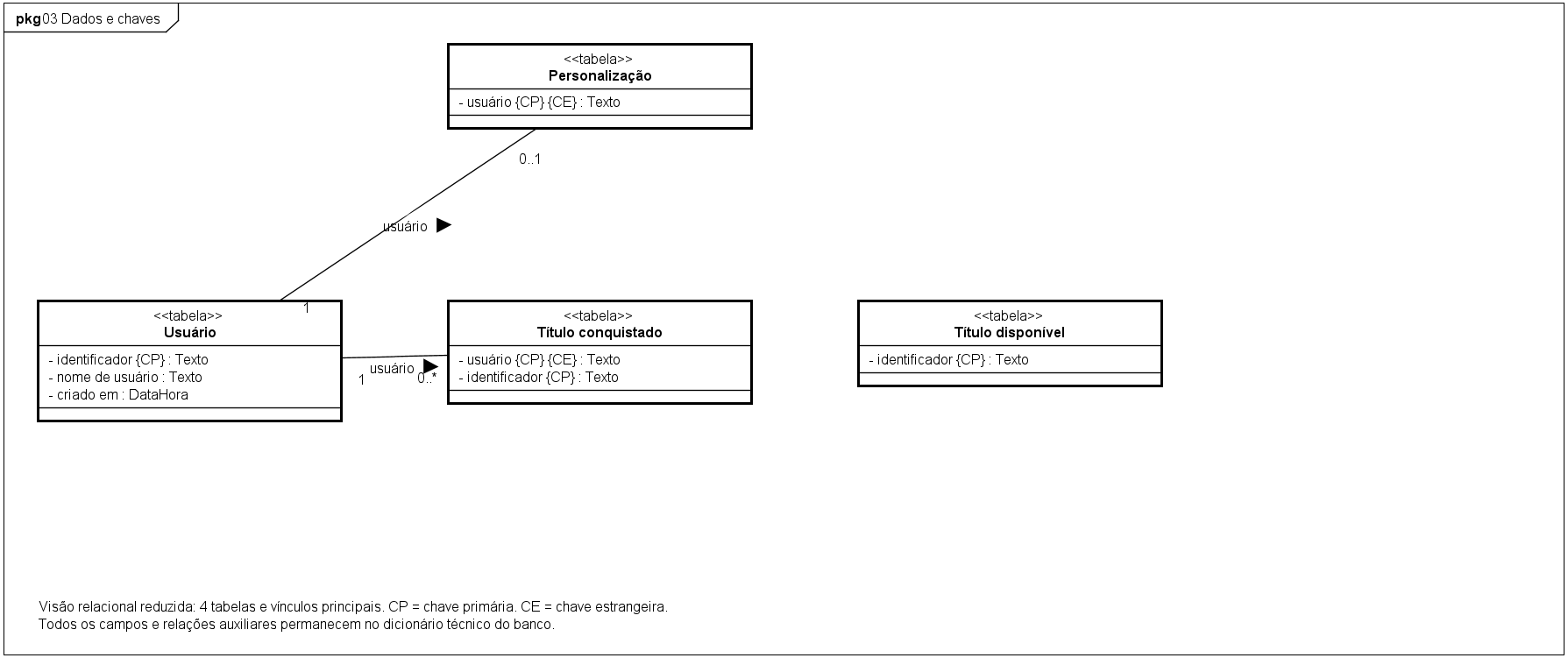
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-IDENTIDADE Classes de identidade

Dados de identidade

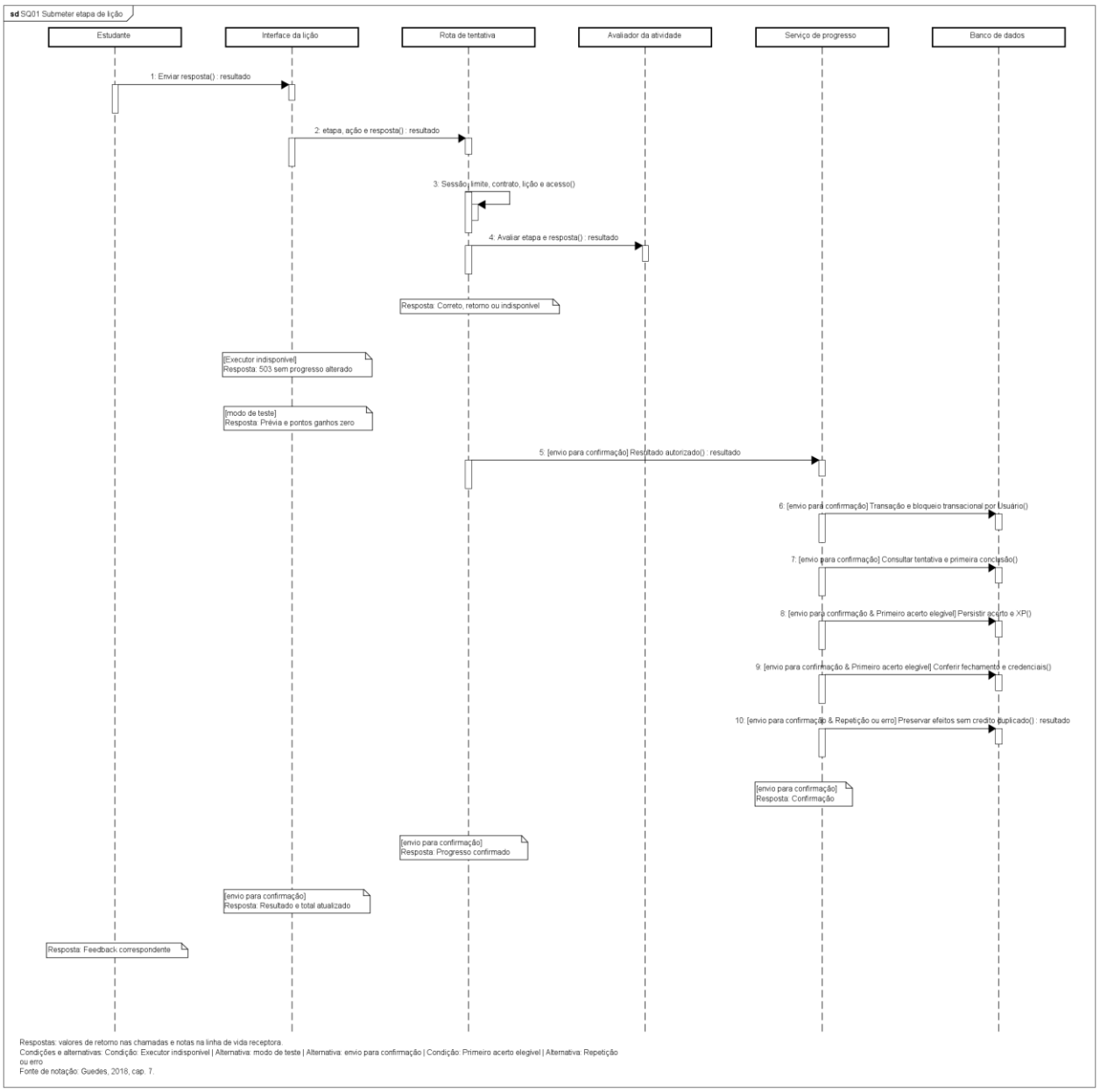
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-IDENTIDADE Dados de identidade

Sequência completa 1

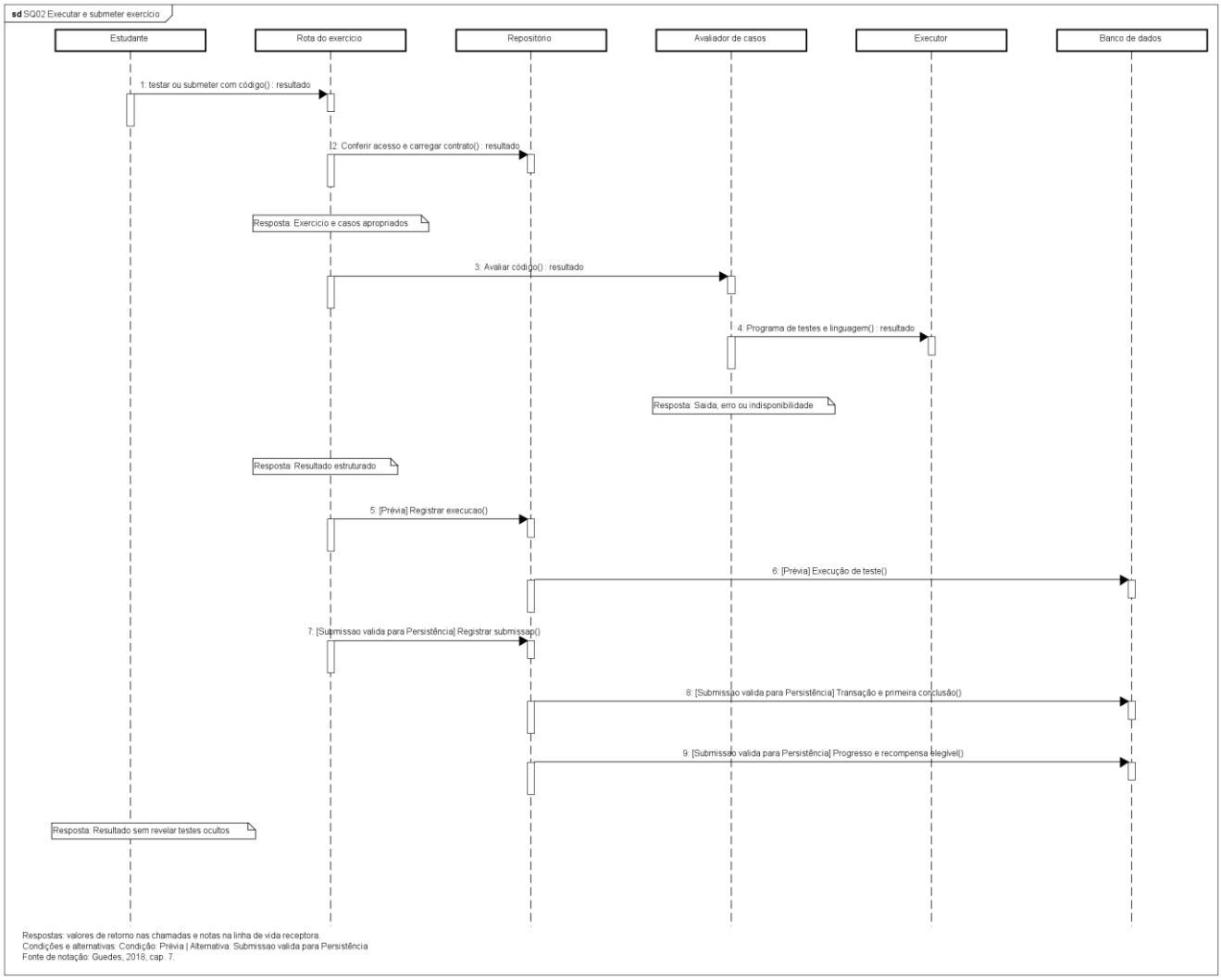
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ01 Sequência completa 1

Sequência completa 2

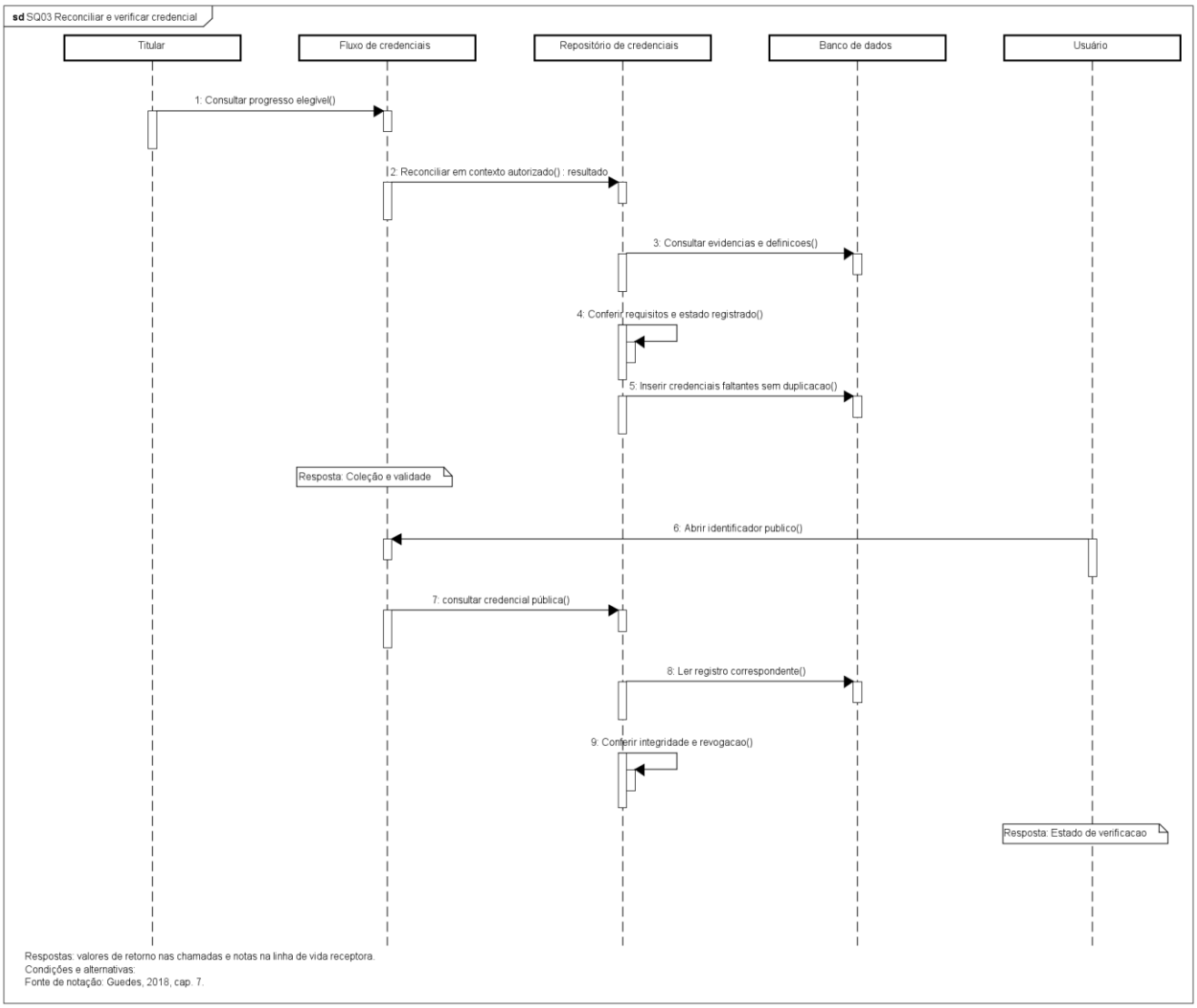
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ02 Sequência completa 2

Sequência completa 3

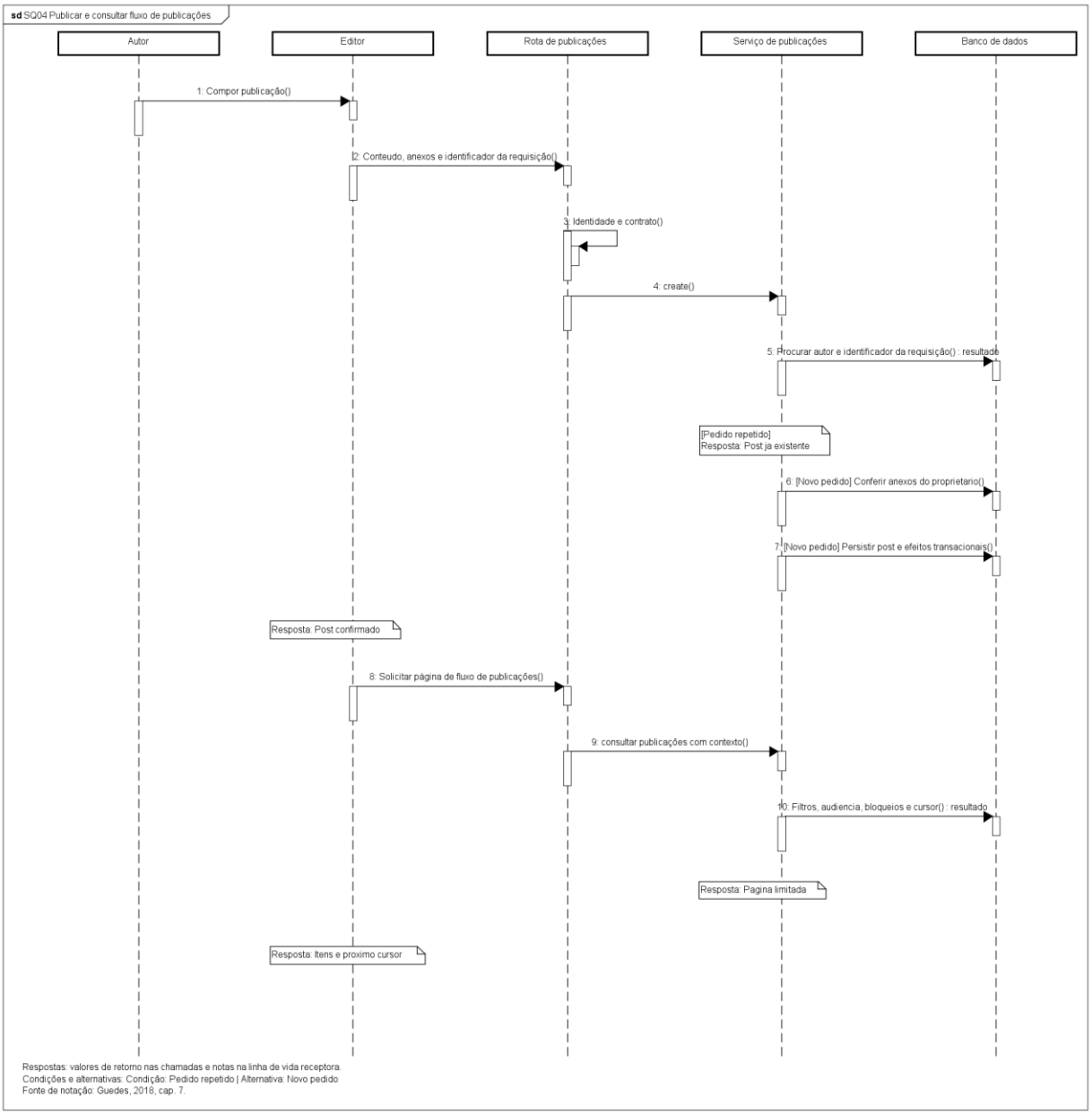
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ03 Sequência completa 3

Sequência completa 4

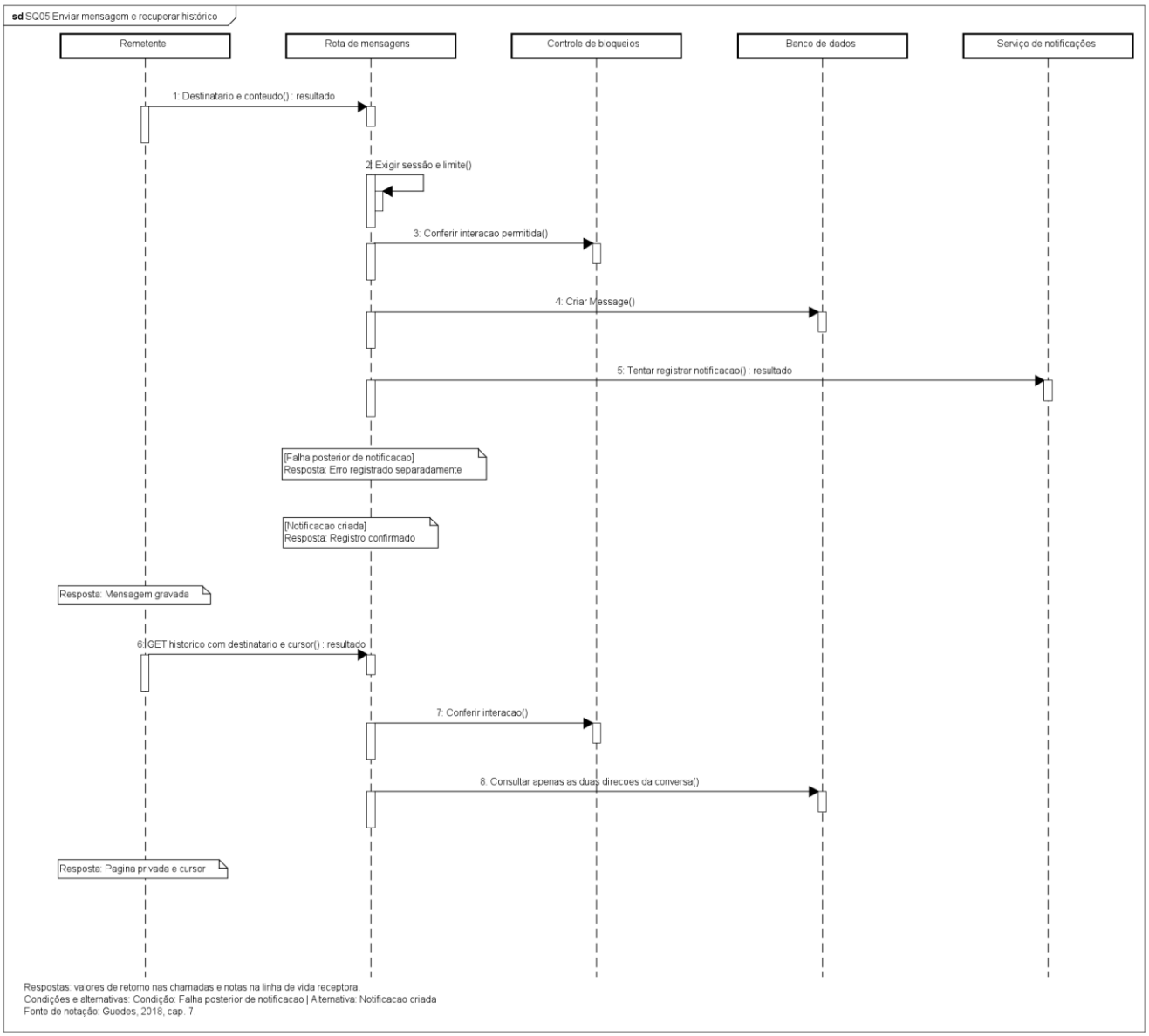
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ04 Sequência completa 4

Sequência completa 5

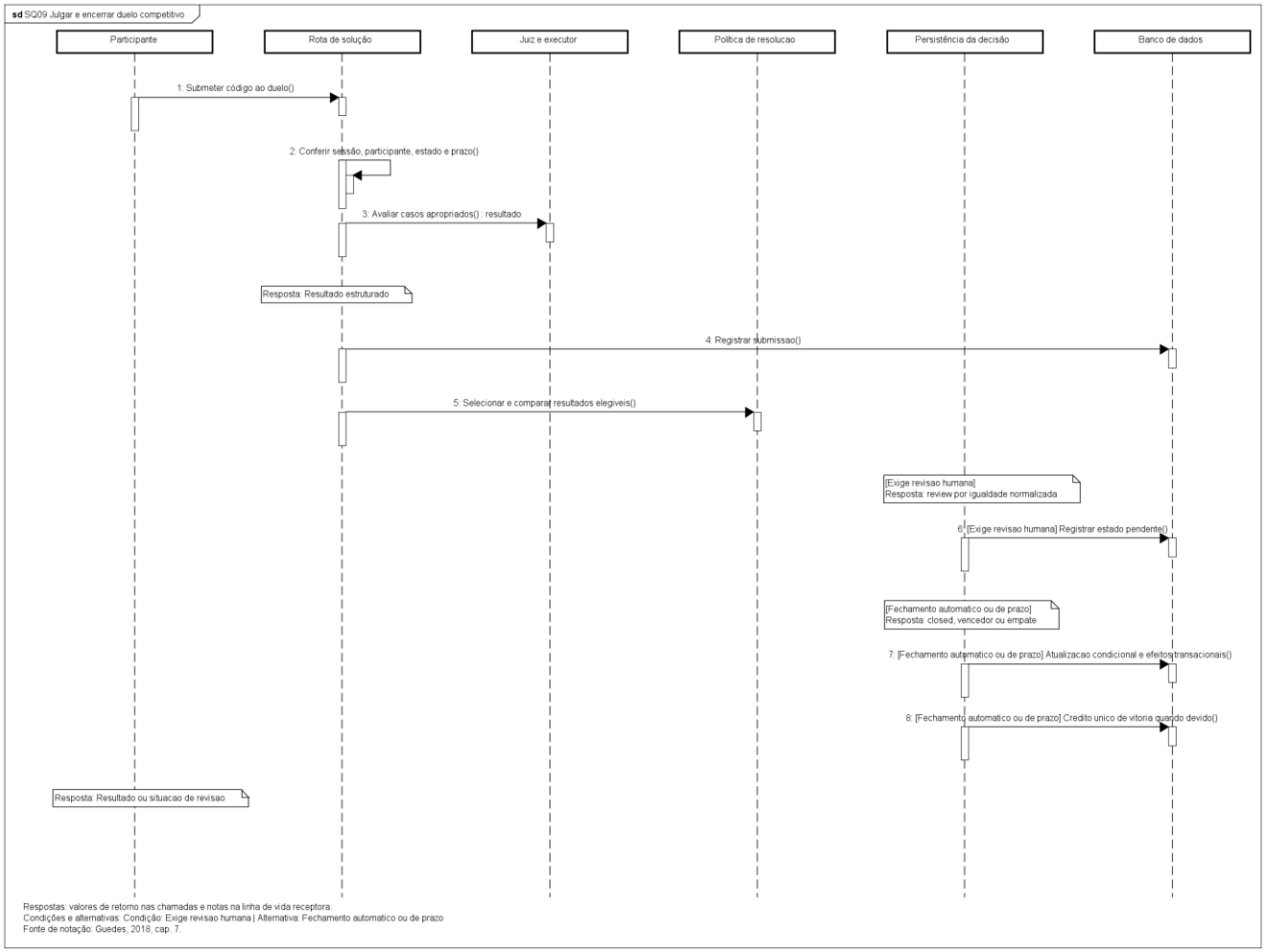
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ05 Sequência completa 5

Sequência completa 9

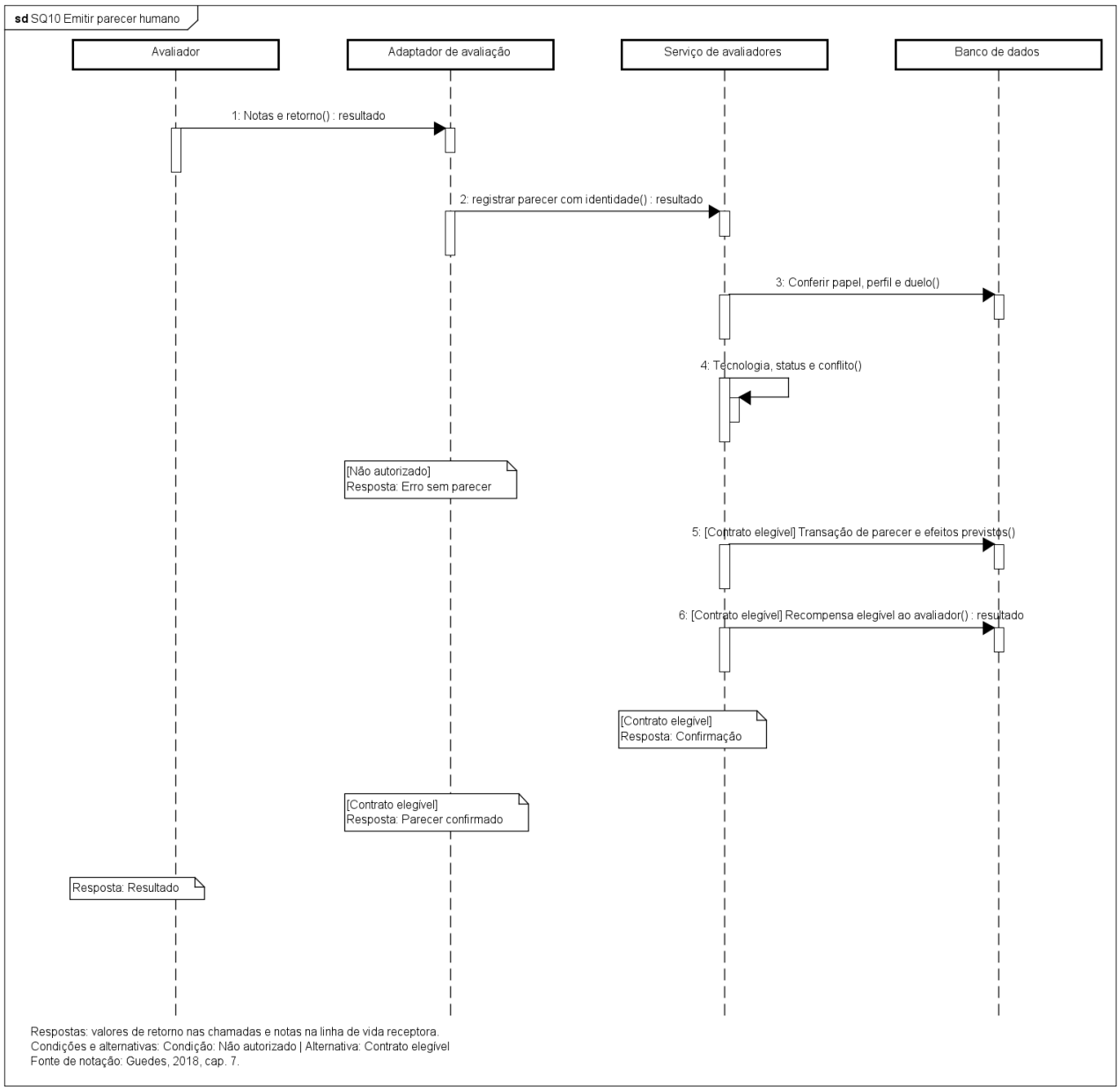
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ09 Sequência completa 9

Sequência completa 10

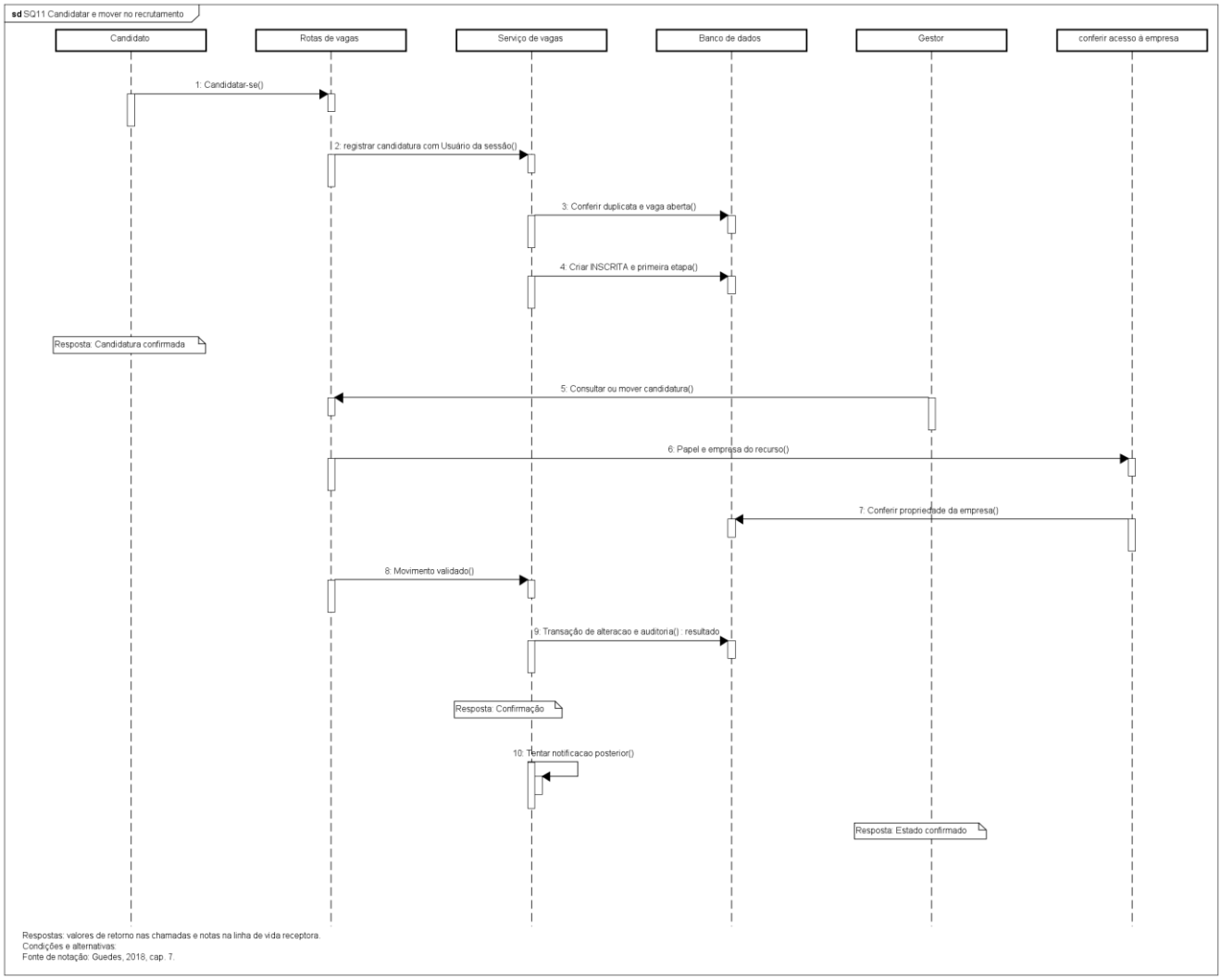
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ10 Sequência completa 10

Sequência completa 11

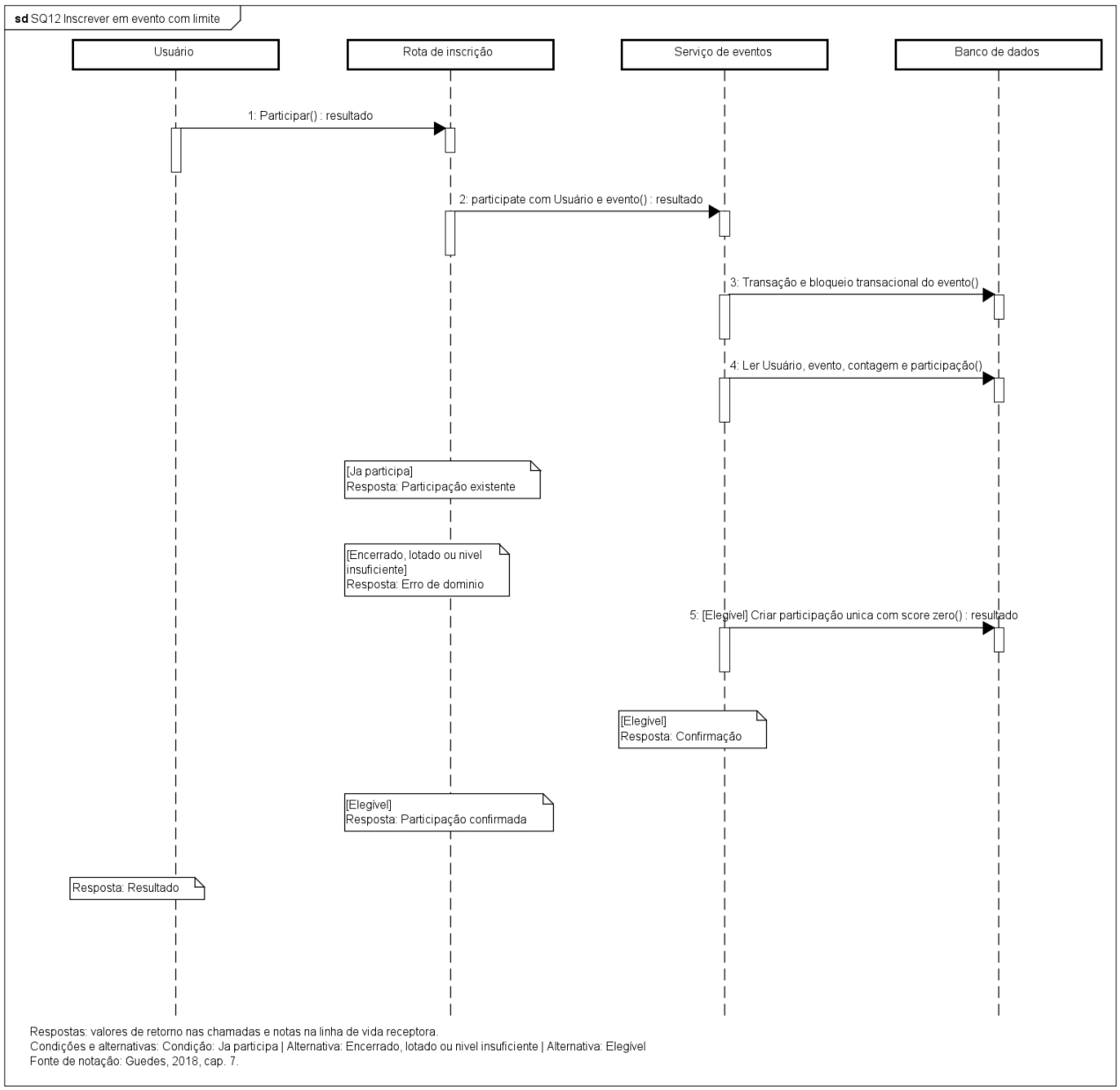
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11 Sequência completa 11

Sequência completa 12

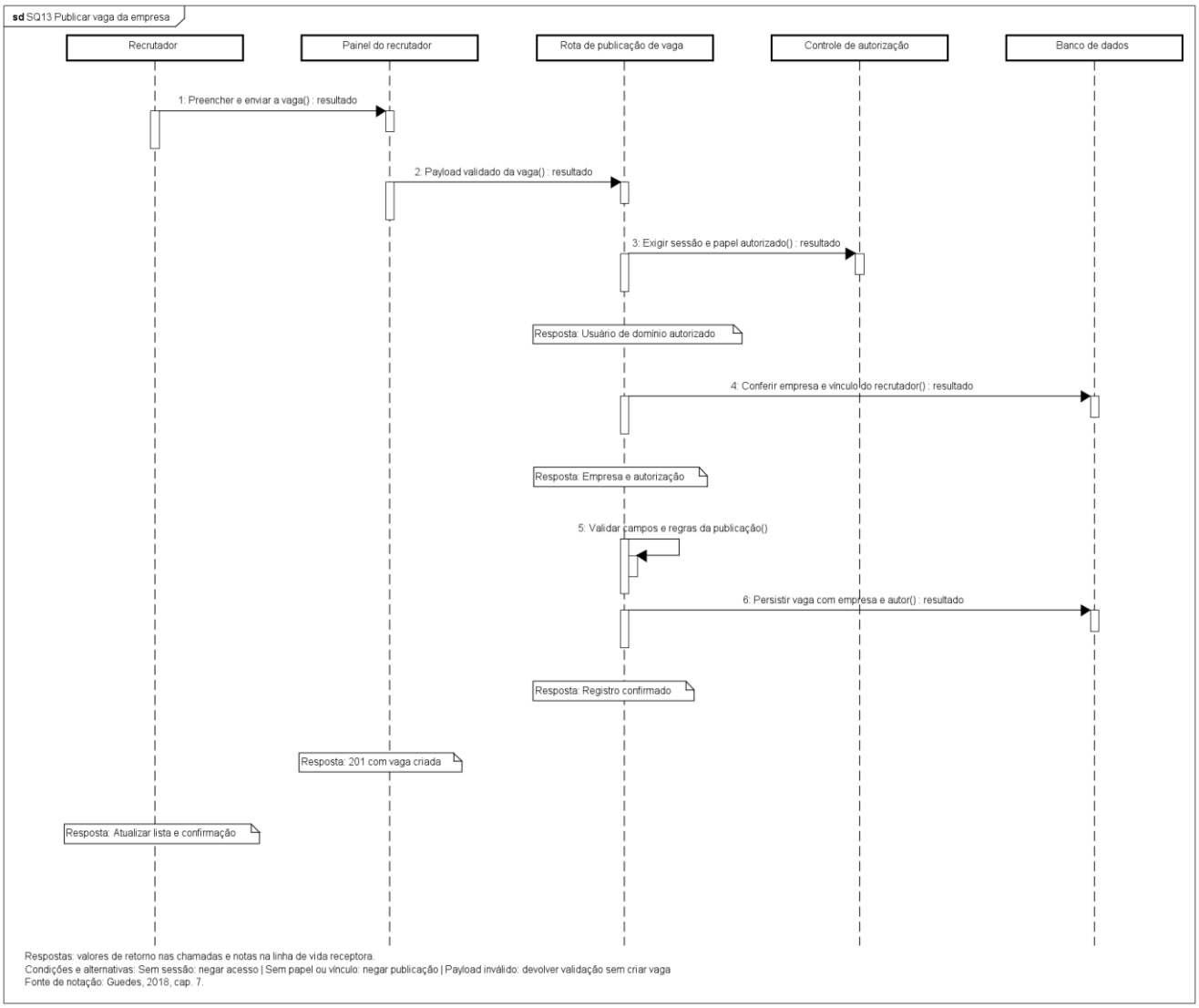
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ12 Sequência completa 12

Sequência completa 13

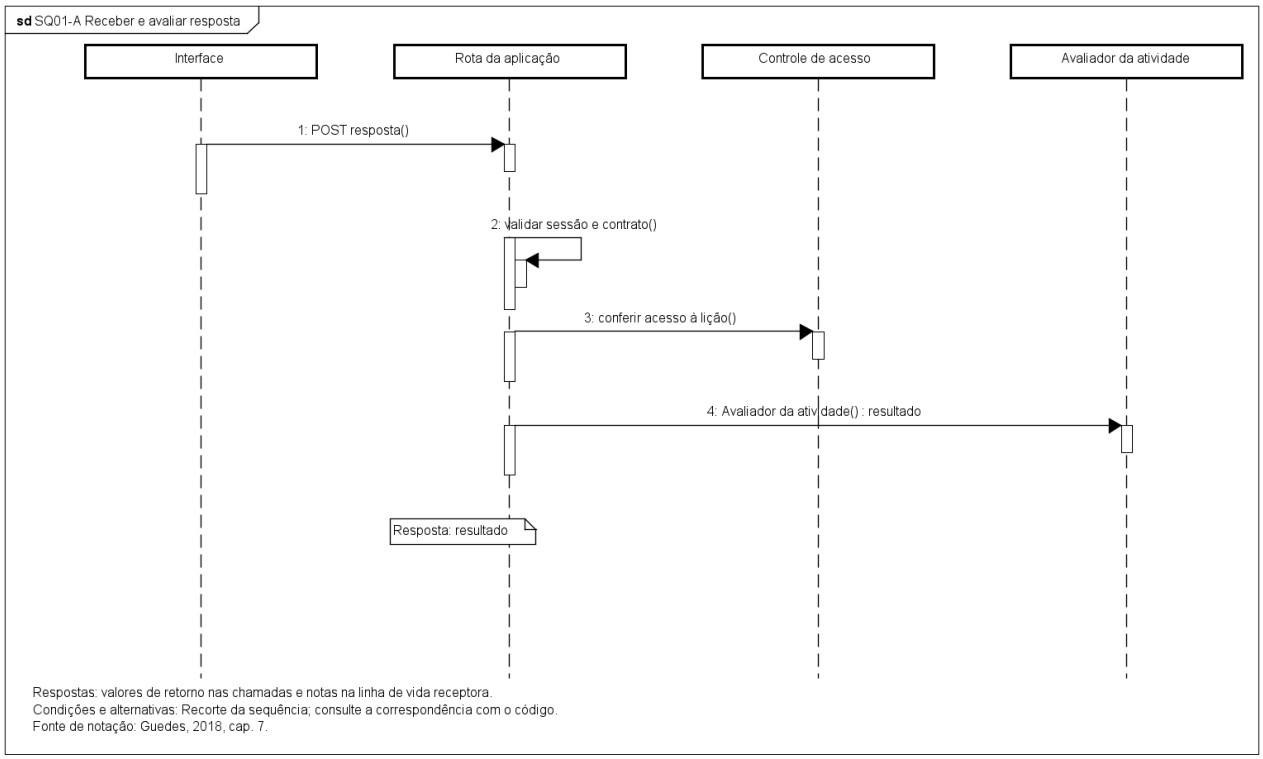
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ13 Sequência completa 13

Receber e avaliar resposta

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



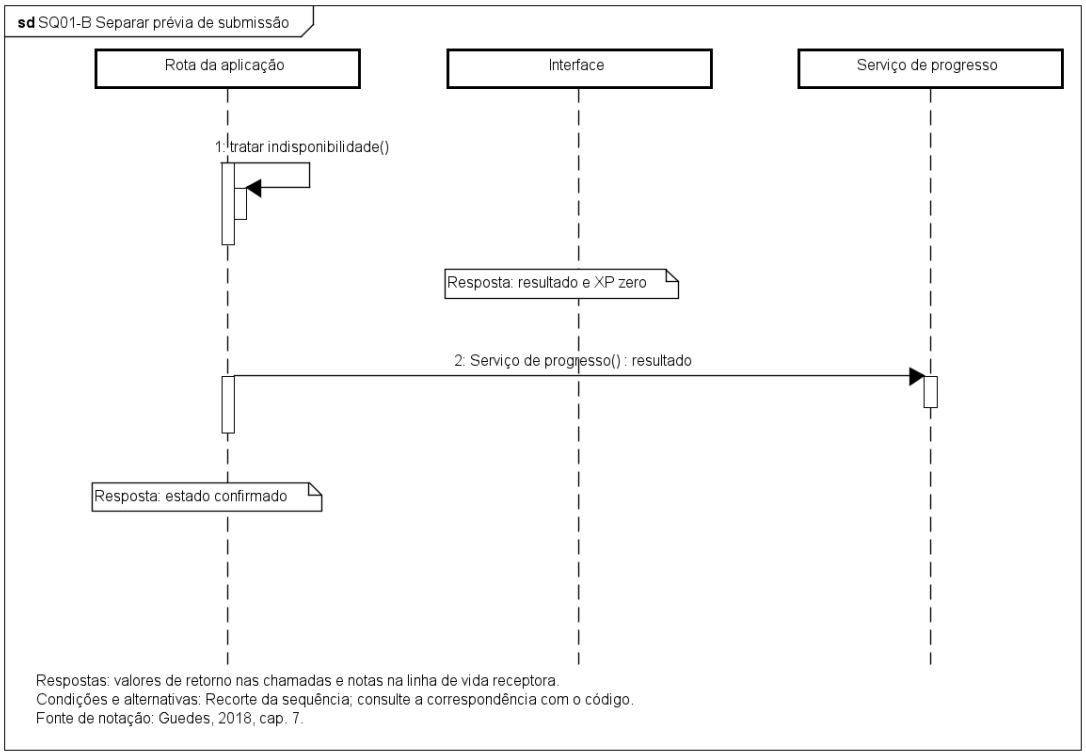
SQ01-A Receber e avaliar resposta

Código real: src/app/api/lessons/[lessonId]/attempt/route.ts, linhas 10 a 31. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
export const POST = apiHandler(async (request, { params, session: user }) => {
  if (!user) throw new UnauthorizedError();
  const { lessonId } = await params;
  await rateLimit(`lesson:${user.id}`, {
    limit: 30,
    window: '1 m',
    endpoint: '/api/lessons/attempt',
  });
  const parsed = lessonAnswerSchema.safeParse(await request.json().catch(() => null));
  if (!parsed.success)
    throw new ValidationError('INVALID_ANSWER', 'Confira os dados da resposta e tente novamente.');
  const answer = parsed.data;
  const lesson = getLessonById(lessonId);
  const step = lesson?.steps.find((s) => s.id === answer.stepId);
  if (!lesson || !step) throw new AppError('LESSON_NOT_FOUND', 'Atividade não encontrada.', 404);
  await requireLessonAccess(user.id, lesson);
  const outcome = await assessLessonStep(lesson, step, answer);
  if ('unavailable' in outcome && outcome.unavailable)
    throw new AppError(
      'RUNNER_UNAVAILABLE',
      'Serviço de execução indisponível. Seu progresso não foi alterado; tente novamente.',
      503
```

Separar prévia de submissão

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



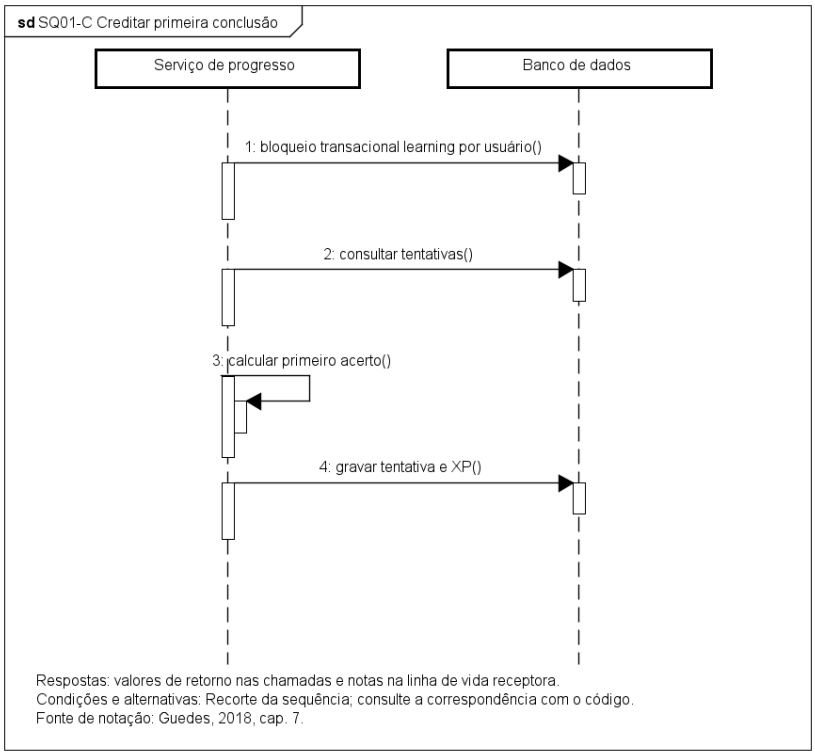
SQ01-B Separar prévia de submissão

Código real: src/app/api/lessons/[lessonId]/attempt/route.ts, linhas 33 a 45. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
  if (answer.action === 'run') return NextResponse.json({ ...outcome, xpEarned: 0 });
  const progress = await saveLessonAssessment(
    user.id,
    lesson,
    step,
    outcome.isCorrect,
    answer.selectedOption ?? 0
  );
  if (outcome.isCorrect) {
    await syncProfileAchievementsSafely(user.id);
  }
  return NextResponse.json({ ...outcome, ...progress });
});
```

Creditar primeira conclusão

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



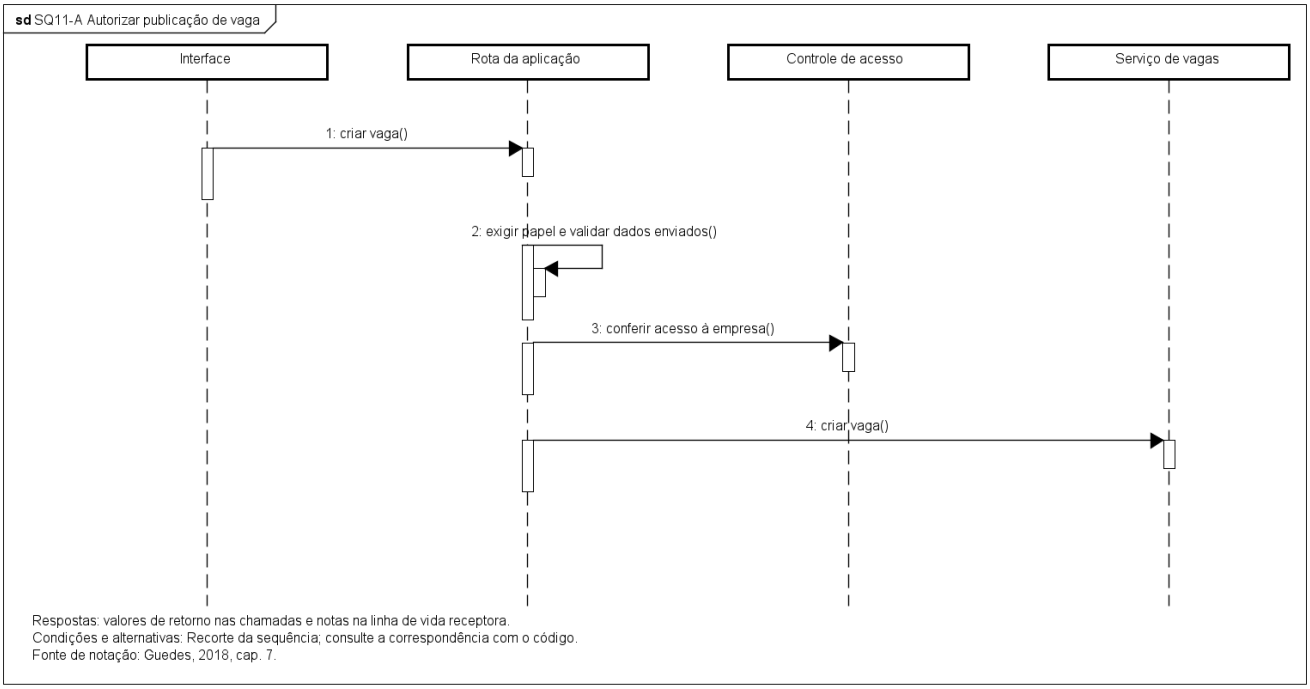
SQ01-C Creditar primeira conclusão

Código real: src/lib/learning/lesson-progress.ts, linhas 62 a 83. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
const firstCompletion = isCorrect && !previous?.is_correct;
const xpEarned = firstCompletion ? Math.max(0, step.xp) : 0;
if (!previous)
  await tx.quiz.upsert({
    where: { id: step.id },
    create: {
      id: step.id,
      question: step.question ?? step.instruction ?? step.title,
      options: step.options ?? ['Avaliado no servidor'],
      correct_index: step.correctOptionIndex ?? 0,
      is_daily: false,
    },
    update: {},
  });
if (!previous)
  await tx.quizAttempt.create({
    data: {
      user_id: userId,
      quiz_id: step.id,
      selected_index: selectedIndex,
      is_correct: isCorrect,
      xp_earned: xpEarned,
```

Autorizar publicação de vaga

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11-A Autorizar publicação de vaga

Código real: src/app/api/jobs/route.ts, linhas 72 a 95. Contexto e ramos omitidos devem ser consultados no arquivo completo.

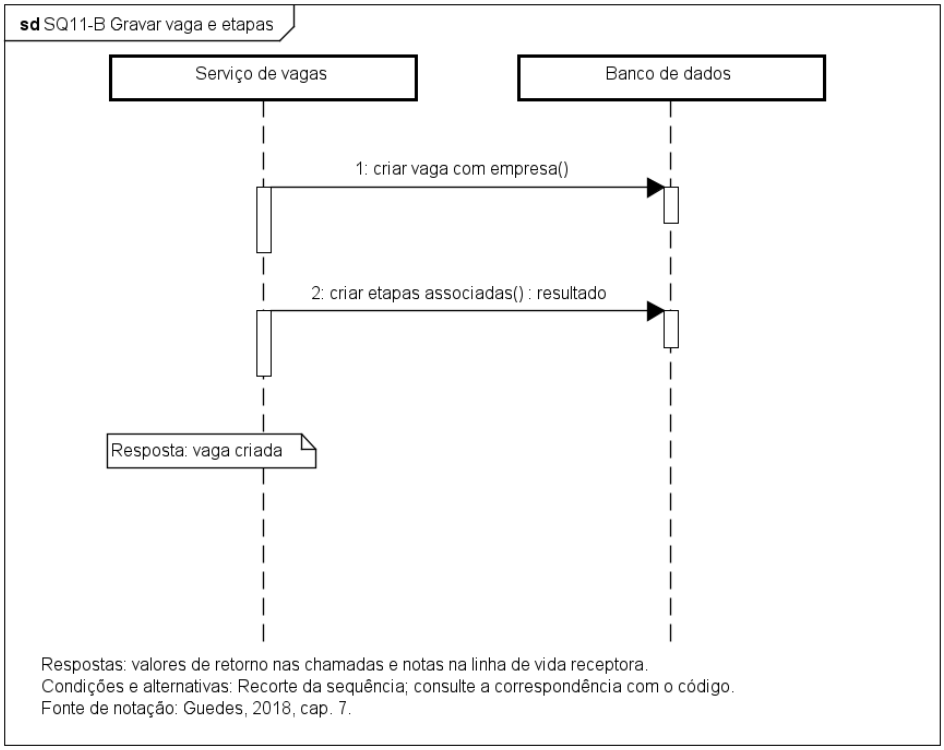
```
export const POST = apiHandler(async (req) => {
  const user = await requireRole(['RECRUITER', 'ADMIN']);
  const body = await req.json();
  const parsed = createJobSchema.parse(body);
  await requireCompanyAccess(parsed.company_id, user);

  const job = await JobService.createJob({
    companyId: parsed.company_id,
    title: parsed.title,
    description: parsed.description,
    level: parsed.level,
    technologies: parsed.technologies,
    modality: parsed.modality,
    location: parsed.location,
    contractType: parsed.contract_type,
    salaryMin: parsed.salary_min,
    salaryMax: parsed.salary_max,
    requirements: parsed.requirements,
    benefits: parsed.benefits,
    stages: parsed.stages,
  });

  return NextResponse.json({
    success: true,
```

Gravar vaga e etapas

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



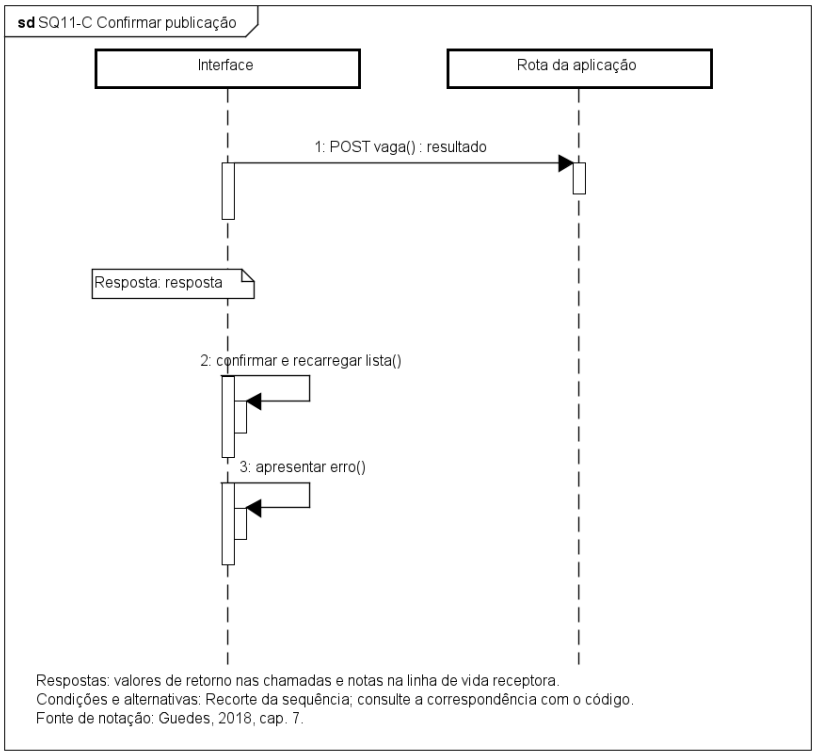
SQ11-B Gravar vaga e etapas

Código real: src/services/job.service.ts, linhas 218 a 247. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
    const job = await prisma.job.create({
      data: {
        company_id: companyId,
        title,
        description,
        level,
        technologies,
        modality,
        location,
        contract_type: contractType,
        salary_min: salaryMin,
        salary_max: salaryMax,
... (linhas intermediárias omitidas neste extrato) ...
            challenge_id: st.challengeId,
            description: st.description,
          })),
        },
      },
      include: {
        stages: true,
        company: true,
      },
    });
```

Confirmar publicação

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



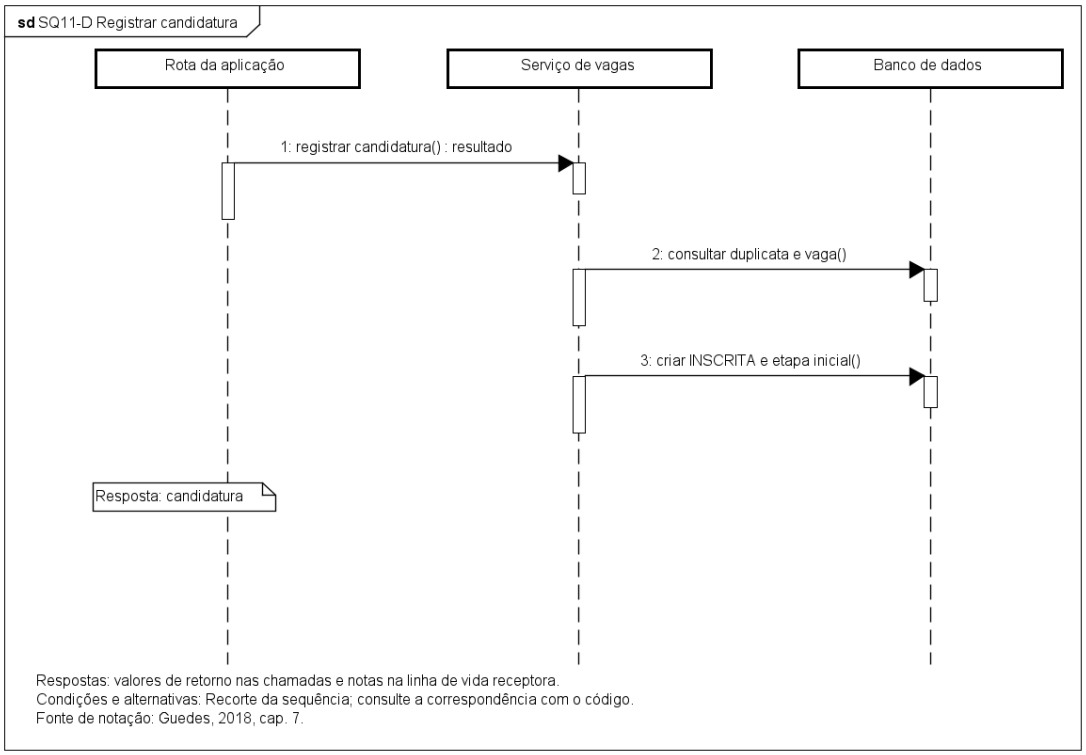
SQ11-C Confirmar publicação

Código real: src/app/recruiter/RecruiterContent.tsx, linhas 378 a 394. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
    if (res.ok) {
      setSuccessMsg('Vaga publicada com sucesso! As etapas do processo já estão disponíveis.');
      setJobTitle('');
      setJobDescription('');
      loadJobs(); // Instantly update jobs list
      setTimeout(() => setSuccessMsg(null), 4000);
    } else {
      const errData = await res.json().catch(() => null);
      setErrorMsg(errData?.message || errData?.error || 'Erro ao publicar vaga.');
    }
  } catch (err) {
    console.error(err);
    setErrorMsg('Erro de conexão ao publicar vaga.');
  } finally {
    setCreatingJob(false);
  }
};
```

Registrar candidatura

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11-D Registrar candidatura

Código real: src/services/job.service.ts, linhas 282 a 304. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
  const firstStage = job.stages[0];

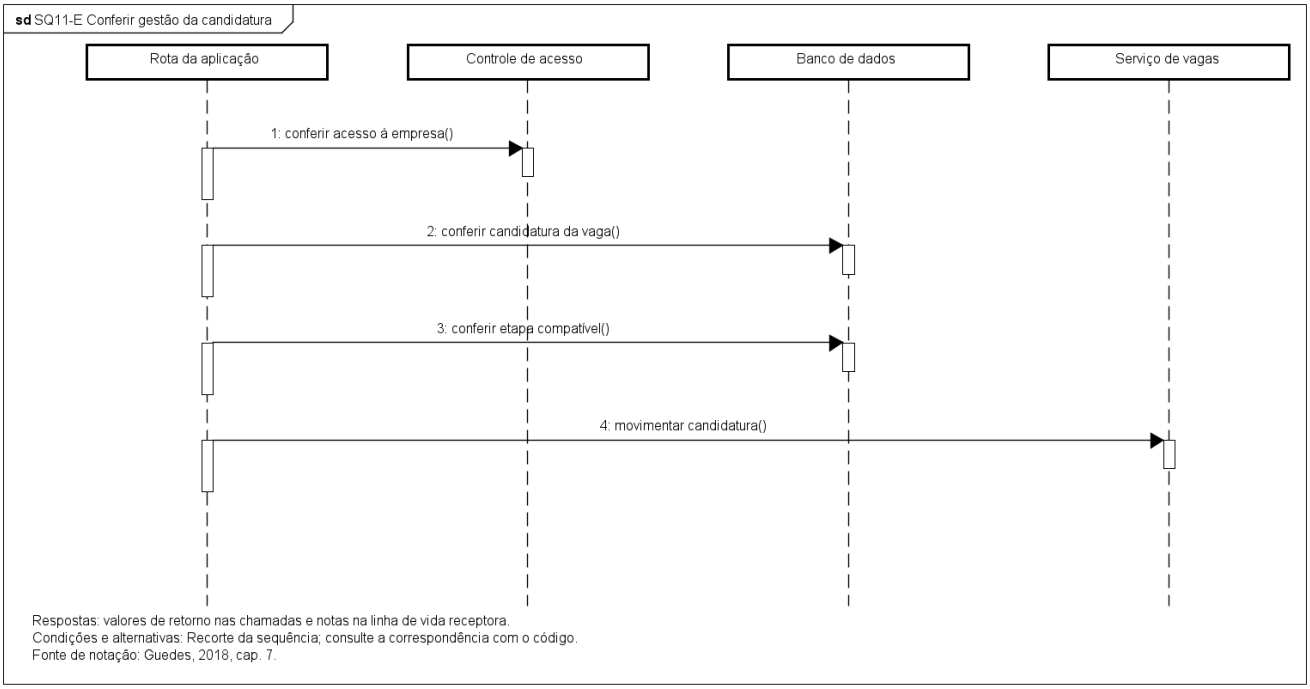
  const application = await prisma.jobApplication.create({
    data: {
      job_id: jobId,
      user_id: userId,
      stage_id: firstStage?.id,
      status: 'APPLIED',
    },
    include: {
      stage: true,
      job: { include: { company: true } },
    },
  });

  logger.info('User applied for job', { userId, jobId, applicationId: application.id });
  return application;
},

/**
 * Advance or update candidate application status in recruitment pipeline.
 */
async updateApplicationStage(params: {
```

Conferir gestão da candidatura

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11-E Conferir gestão da candidatura

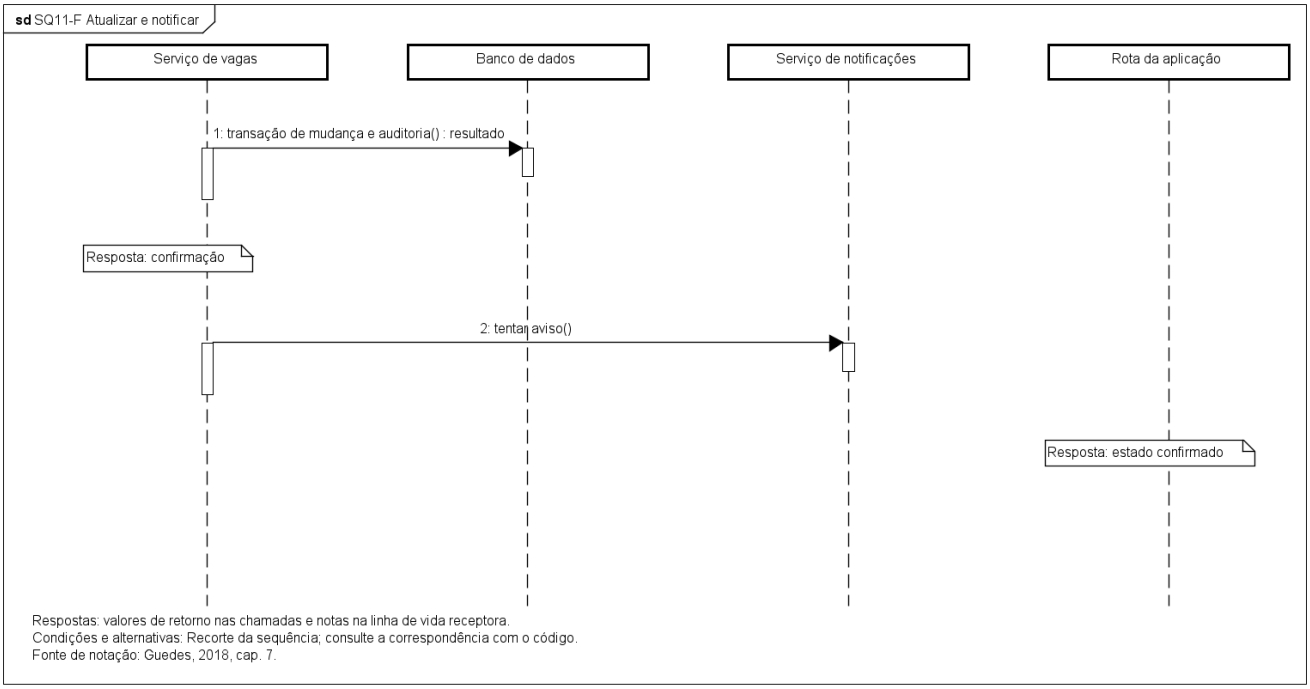
Código real: src/app/api/jobs/[id]/route.ts, linhas 30 a 62. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
export const PATCH = apiHandler(async (req, { params }) => {
  const user = await requireRole(['RECRUITER', 'ADMIN']);
  const { id } = await params;
  const job = await prisma.job.findUniqueOrThrow({ where: { id } });
  await requireCompanyAccess(job.company_id, user);
  const body = await req.json();
  const parsed = updateStageSchema.parse(body);
  const application = await prisma.jobApplication.findFirst({
    where: { id: parsed.application_id, job_id: id },
  });
  if (!application)
    return NextResponse.json({ error: 'Candidatura não encontrada nesta vaga.' }, { status: 404 });
... (linhas intermediárias omitidas neste extrato) ...
    technicalScore: parsed.technical_score,
    actorId: user.id,
  });

  return NextResponse.json({
    success: true,
    message: 'Etapa do candidato atualizada com sucesso!',
    application: updatedApplication,
  });
});
```

Atualizar e notificar

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11-F Atualizar e notificar

Código real: src/services/job.service.ts, linhas 314 a 359. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
    const application = await prisma.$transaction(async (tx) => {
      const updated = await tx.jobApplication.update({
        where: { id: applicationId },
        data: {
          stage_id: stageId,
          status,
          feedback,
          technical_score: technicalScore,
        },
        include: {
          stage: true,
          user: {
... (linhas intermediárias omitidas neste extrato) ...
        type: 'JOB_APPLICATION_UPDATE',
        resourceId: application.job_id,
        resourceType: 'JOB',
      });
    } catch (err) {
      logger.error('Failed to notify job application update', { error: String(err) });
    }

    return application;
  },
```

Correspondência dos modelos com a implementação

Os nomes em português são conceitos e responsabilidades do domínio. As operações conceituais não afirmam que exista uma função TypeScript com a mesma grafia. Esta tabela localiza os modelos e os contratos reais para a leitura do código.

Classe no diagrama	Referência técnica
Usuário	User
Trilha	src/lib/learning
Lição	src/lib/learning/catalog.ts
Atividade	Quiz / Exercise
Tentativa	QuizAttempt / ExerciseSubmission
Credencial	LearningCredential
Duelo	Duel
Solução	DuelSubmission
Parecer	DuelEvaluation
Perfil de avaliador	EvaluatorProfile
Publicação	Post
Comentário	Answer
Perfil	User / ProfileCustomization
Empresa	Company
Vaga	Job
Etapa da seleção	JobRecruitmentStage
Candidatura	JobApplication
Evento	Event
Inscrição	EventParticipant
Desafio de evento	EventChallenge
Mensagem	Message
Notificação	Notification
Preferência	User metadata / MobileState

Tabelas relacionais

Nome em português	Modelo Prisma
Usuário	User
Progresso da linguagem	LanguageTrail
Atividade	Quiz
Tentativa	QuizAttempt
Credencial	LearningCredential
Publicação	Post
Comentário	Answer
Mensagem	Message
Notificação	Notification
Empresa	Company
Vaga	Job
Etapa da seleção	JobRecruitmentStage
Candidatura	JobApplication
Duelo	Duel
Solução enviada	DuelSubmission
Parecer	DuelEvaluation
Perfil de avaliador	EvaluatorProfile
Evento	Event
Inscrição	EventParticipant
Desafio de evento	EventChallenge
Personalização	ProfileCustomization
Título conquistado	UserProfileAchievement
Título disponível	ProfileAchievementCatalog

Uso da documentação no seminário

O que foi aproveitado do material fornecido

A lousa e a transcrição organizam a apresentação em duas escalas: visão geral uma vez e explicação específica por participante. Cada recorte deve ligar interface, requisitos, regras, modelagem, sequência, código e demonstração. A entrega escrita deve cobrir também funções que não caibam na exposição oral.

A entrega anterior identifica Pedro Antonio Silvestre Tescaro, Guilherme Alves de Campos e Paulo Henrique Leal dos Santos, nessa ordem proposta para aprendizagem, publicação de vaga e candidatura. Esta revisão conserva essa continuidade, com referência à entrega de 20/10/2026; não presume uma nova distribuição comunicada pelo grupo.

Ordem lógica sugerida

Bloco	Objetivo	Material e preparação
Abertura	Explicar problema, atores e jornada	Visão do sistema, atividade do negócio e panorama de casos
Identidade e arquitetura	Explicar como tela chama servidor e quem pode agir	Guia de programação, classes de responsabilidade e DER geral
Aprendizagem	Demonstrar o núcleo da proposta	Curso, lição, etapa, run, submit, progresso e XP
Reconhecimento e comunidade	Mostrar como evidência e identidade aparecem	Perfil, título, credencial, post e notificação
Competição	Diferenciar juiz de duelo e combate RPG	Duelo ou Mundinho/PvP com dois participantes preparados
Oportunidades	Consumir perfis e progresso em outro contexto	Empresa, vaga, candidatura e funil; evento se houver tempo
Fechamento	Mostrar limites e decisões de engenharia	Atomicidade, proteção de dados e evidências de testes

Os blocos podem ser repartidos entre integrantes. A sequência evita demonstrar uma candidatura antes de existir vaga, ou revisão humana antes de existir avaliador e duelo elegível.

Roteiro de cada funcionalidade

1. Abra a tela e explique quem a utiliza e para quê.

2. Aponte os requisitos funcionais, as propriedades de qualidade e as regras relevantes pelos IDs deste manual.

3. Mostre as responsabilidades de código e as tabelas daquele recorte.

4. Percorra a sequência completa sem mergulhar em todas as linhas.

5. Escolha dois ou três trechos reais e explique entrada, validação, autorização, decisão e efeito.

6. Execute a ação demonstrada e confira o resultado persistido visível.

7. Mostre uma alternativa importante: repetição, bloqueio, resposta incorreta ou falta de requisito.

Não apresente somente um arquivo de serviço sem explicar a interface e o objetivo. Também não apresente somente botões sem explicar como o servidor confirma a regra.

Diagramas gerais e específicos

O modelo de responsabilidades [modelagem.md] não pretende fingir que todos os serviços são classes JavaScript: vários são objetos literais e funções. Explique essa correspondência ao apresentar uma visão de classes conceitual.

O DER deriva do schema e contém relações com chaves. MER é o modelo de entidades e relacionamentos; DER é uma representação diagramática. A presença de FK depende do nível conceitual/lógico/físico escolhido, e não é por si uma definição universal que separa as siglas. Para atender à organização da disciplina, use a visão de banco com PK/FK e explique seus vínculos, evitando duplicar slides sem informação nova.

Recortes de sequência e código

Demonstração	Trechos especialmente úteis
Lição	Rota de tentativa, assessLessonStep, saveLessonAssessment
Exercício	Avaliador, execução e recordExerciseSubmission
Baú	dayKey, credit, ramo da ação chest
PvP	chooseArenaAttack, advanceArenaTurn, conferências em arena
Duelo	Política decideDuelResolution e persistência de resultado
Vaga	applyForJob, requireCompanyAccess, updateApplicationStage
Evento	participate e lock de capacidade
Perfil/credencial	Conciliação de conquista versus emissão de credencial

Os recortes gerados [codigo-comentado.md] conservam origem e linhas; o arquivo completo deve estar disponível para responder perguntas.

Dados preparados

Prepare contas de demonstração e papéis adequados. Use atividades e objetos controlados: lição com resposta conhecida, outro usuário para interação, vaga aberta, evento em datas válidas e adversário disponível. Se uma informação não será explicada, pode estar previamente criada, mas diga que é dado preparado.

Não prometa que um compilador externo estará disponível sem plano alternativo. Uma evidência gravada pode apoiar a explicação, desde que seja identificada como evidência de outra execução e não apresentada como resultado ao vivo.

Tempo e divisão

O material fornecido menciona até uma hora. Esse é o contexto informado, não uma regra técnica do produto. Distribua o tempo por complexidade e dependência; reserve espaço para mudança de conta, carregamento e perguntas. Evite um integrante consumir a abertura inteira e outro mostrar apenas uma tela.

Todos precisam conseguir explicar a teoria e o código de seu recorte, inclusive quem trabalhou principalmente na documentação. O objetivo do encadeamento é permitir essa compreensão sem decorar o projeto inteiro linha por linha.

Checklist de entrega

Casos de uso escritos cobrem todos os módulos, não só os escolhidos para os slides.

Diagramas gerais e vistas específicas estão coerentes com a mesma base.

Regras citadas têm fonte e não confundem desejo futuro com implementação.

Sequências terminam na confirmação/erro, não apenas na chamada à API.

Dados de demonstração não envolvem membros reais sem autorização.

Credenciais e capturas não expõem segredos ou dados privados.

Relatórios antigos conservam sua data; esta edição não os declara atualizados automaticamente.

O ensaio permite executar início, resultado e pelo menos uma alternativa importante.

As entregas históricas de Word, PDF, slides e Astah permanecem no diretório da disciplina [../../laboratorio-engenharia-software/README.md]. Esta edição é a documentação Markdown atualizada que pode orientar uma revisão futura desses artefatos.

Leitura do diagrama de classes geral

A vista geral apresenta Usuário, Empresa, Vaga, Candidatura, Publicação, Tentativa, Duelo, Evento, Inscrição e Mensagem. O primeiro compartimento contém o nome, o segundo os atributos privados e o terceiro as operações públicas. As multiplicidades indicam quantas instâncias participam de cada associação.

Os identificadores UUID e os textos usam string; contagens e pontuações usam int; resultados lógicos usam boolean. DateTime representa data e hora, void indica ausência de retorno e List<T> representa uma coleção tipada. Os nomes de domínio permanecem em português.

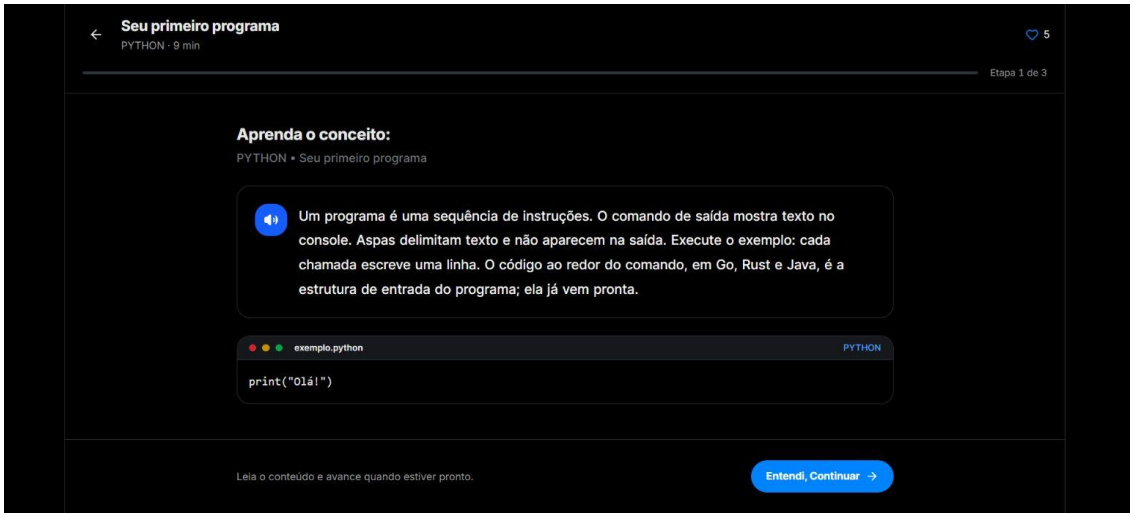
As operações sintetizam responsabilidades do domínio. A implementação distribui essas responsabilidades entre serviços, rotas e bibliotecas; os registros Prisma não se tornam classes TypeScript com métodos por causa desta representação.

Classe geral	**Modelo de dados correspondente**
Usuário	User
Empresa	Company
Vaga	Job
Candidatura	JobApplication
Publicação	Post
Tentativa	QuizAttempt
Duelo	Duel
Evento	Event
Inscrição	EventParticipant
Mensagem	Message

Interfaces atuais dos fluxos apresentados

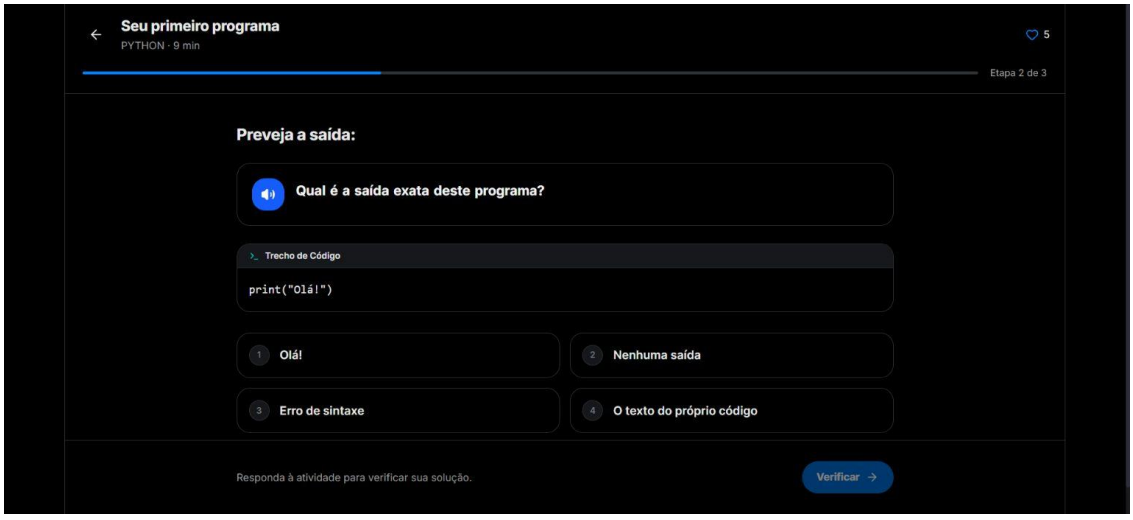
Capturas realizadas em 3 de novembro de 2026 na aplicação web Stacklyst. Cada imagem apresenta o estado carregado da tela consultada. Dados, posições e disponibilidade podem mudar com novas atividades.

Realizar Lição



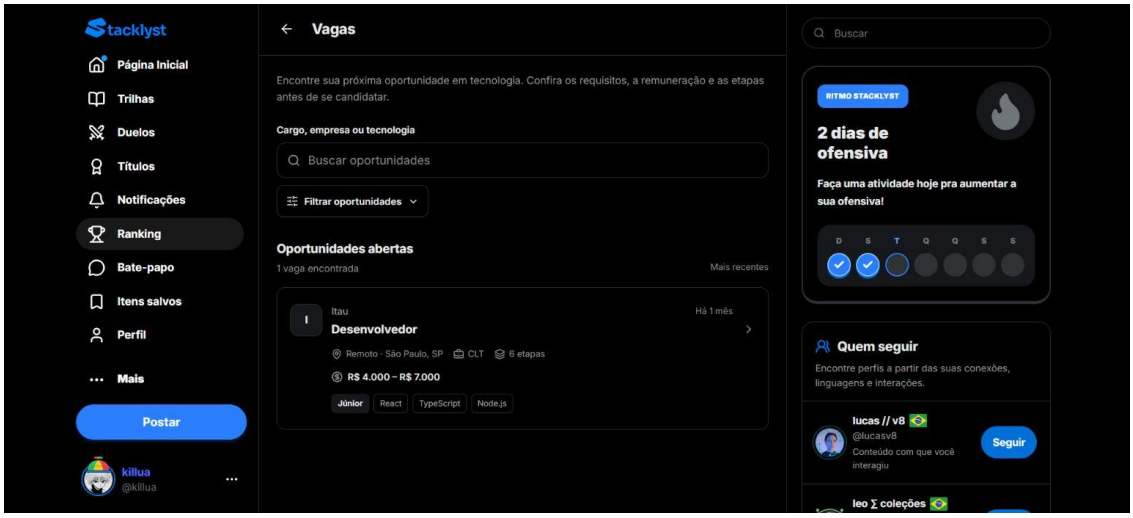
Realizar Lição — https://stacklyst.vercel.app/lesson/learn-python-first-output?returnTo=%2Ftrails%3Fview%3Dmap%26path%3Dfoundations-python%26language%3DPYTHON

Resolver exercício



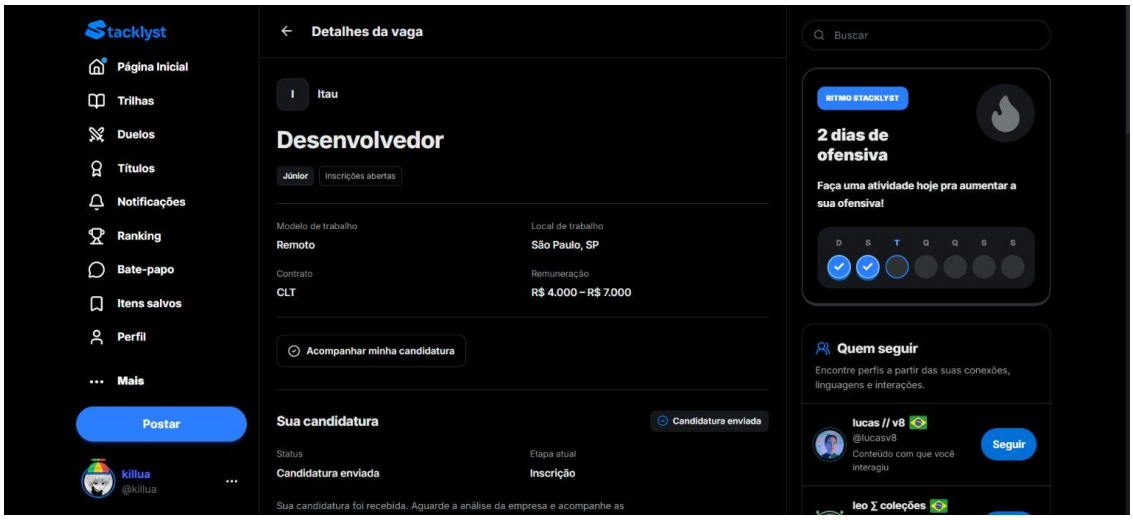
Resolver exercício — https://stacklyst.vercel.app/lesson/learn-python-first-output?returnTo=%2Ftrails%3Fview%3Dmap%26path%3Dfoundations-python%26language%3DPYTHON

Vagas e recrutamento



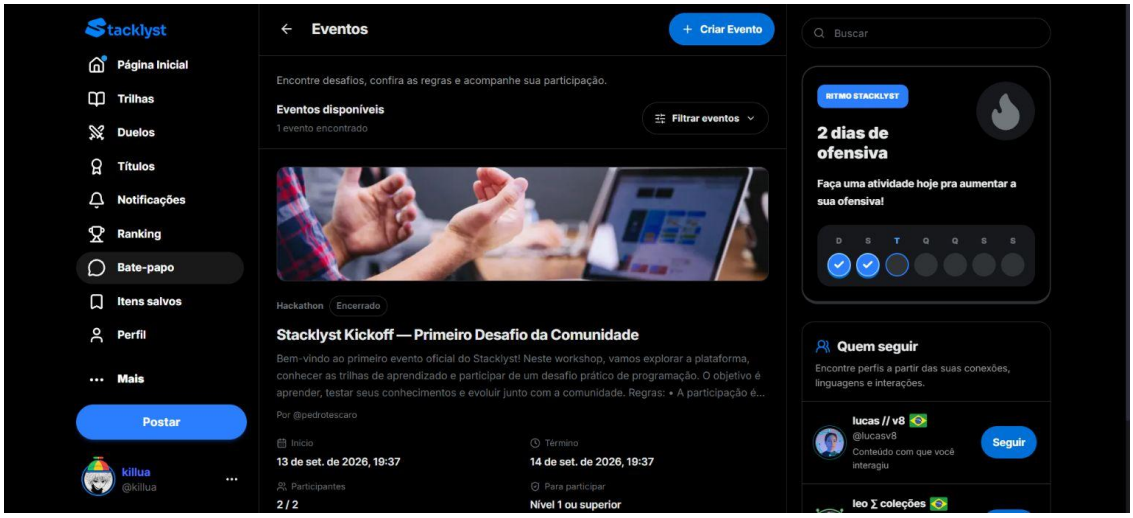
Vagas e recrutamento — https://stacklyst.vercel.app/jobs

Detalhes da vaga e acompanhamento da candidatura



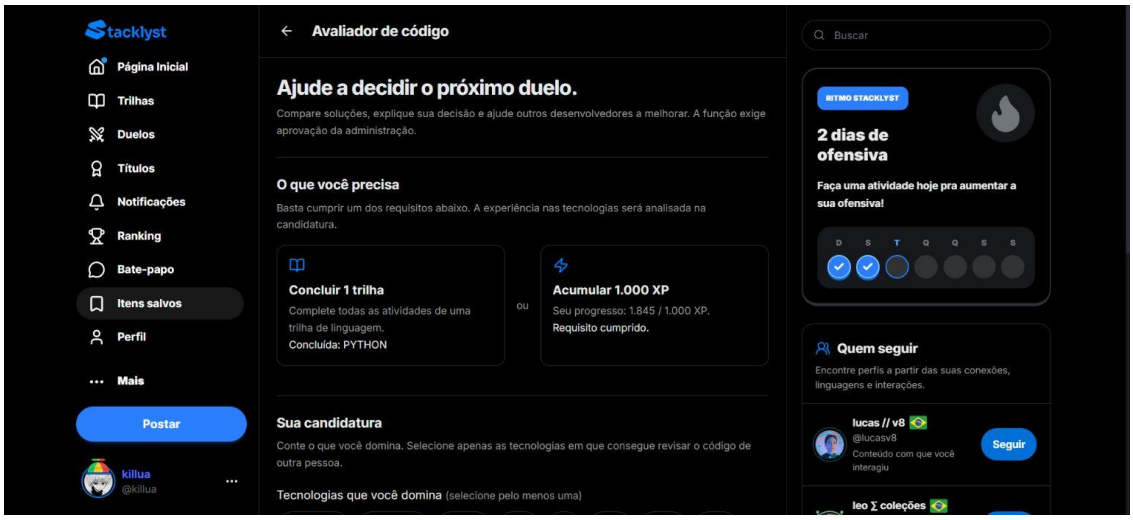
Detalhes da vaga e acompanhamento da candidatura — https://stacklyst.vercel.app/jobs/43827ed5-dbbc-4256-9c63-a16dbd6c9d58

Eventos e hackathons



Eventos e hackathons — https://stacklyst.vercel.app/events

Avaliação de código e pedido de credenciamento



Avaliação de código e pedido de credenciamento — https://stacklyst.vercel.app/evaluators/apply